

— A FIELD GUIDE TO THE CONTROL PLANE

Inside Kubernetes

Following a Deployment from Your Terminal
to a Running Application

A practical journey through the Kubernetes control plane —
exploring what really happens after you run `kubectl apply`, and
why understanding the internals makes you a better engineer.

● LIVING DOCUMENT

EDITION 1.0

ALL 21 CHAPTERS • PARTS I-VIII

One deployment, end to end

Every chapter in this guide is one station on the path below. This is the map we keep returning to — from a command typed in a terminal to a response served back to a client.

FIGURE 0 · THE REQUEST LIFECYCLE (CHAPTER 21)

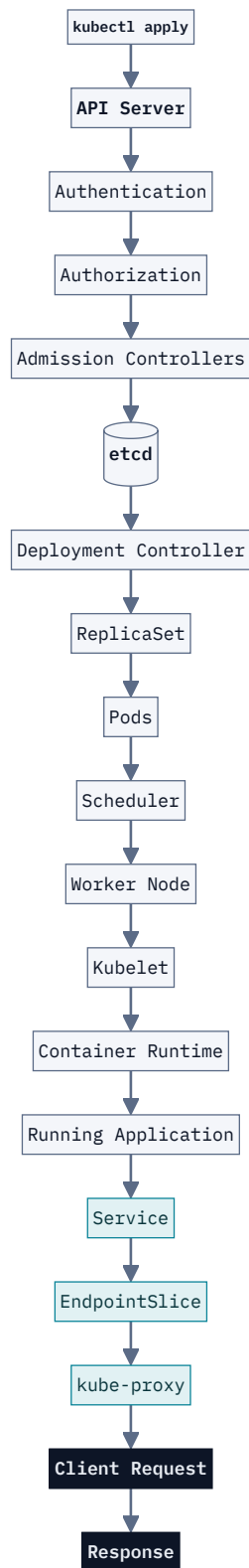


Table of Contents

Chapters fill in as the guide grows. Titles link ahead to their sections once written.

FM

Preface

§

Preface

Why I Wrote This Guide · Who This Guide Is For · Why Understanding Kubernetes Still Matters in the Age of AI · How to Read This Guide

PART I

The Journey Begins

01

What Really Happens When You Run `kubectl apply`?

The 2:17 AM Production Incident · The Problem with Learning Kubernetes Component by Component · A Different Way to Learn Kubernetes · Our Journey Through the Control Plane · Desired State: The One Concept Everything Builds Upon

PART II

Entering the Cluster

02

The Kubernetes API Server: The Front Door to the Cluster

Why Every Request Starts Here · Receiving the Request · Authentication · Authorization · Admission Controllers · Mutating vs Validating Admission Controllers · Why Everything Goes Through the API Server · Why Components Don't Talk Directly to Each Other

03

etcd: Kubernetes' Source of Truth

What Is etcd? · Why Kubernetes Needs a Distributed Key-Value Store · What Happens After Your Deployment Is Accepted · What Actually Gets Stored? · Desired State vs Current State · Why Nothing Is Running Yet

04 Understanding Controllers

Why They're Called Controllers · The Reconciliation Loop · Desired State vs Actual State · Event-Driven Architecture · Watch API · Informers · Local Cache · Why Kubernetes Doesn't Continuously Poll

05 The Deployment Controller

Discovering a New Deployment · Creating the First ReplicaSet · Why Deployments Never Create Pods Directly · Handing Responsibility to the ReplicaSet

06 ReplicaSets: Keeping Your Application Alive

The Desired Replica Count · Creating Pod Objects · Why Pods Aren't Running Yet · What Happens When a Pod Dies? · Continuous Reconciliation

07 Pods Waiting for a Home

Pending Pods · Why Node = None · Pod Lifecycle Before Scheduling

08 The Kubernetes Scheduler

Watching the API Server · Filtering Nodes · Scoring Nodes · Binding Pods · Resource Requests · Node Affinity · Taints and Tolerations · Topology Spread Constraints · Why Scheduling Is More Than Picking a Node

09 The Kubelet

The Worker Node Agent · How the Kubelet Watches the API Server · Pod Synchronization · Pulling Images · Creating Containers · Mounting Volumes · Configuring Networking · Starting Containers · Reporting Status Back to the API Server

10 Container Runtimes

Why Kubernetes Doesn't Run Containers · CRI (Container Runtime Interface) · containerd · CRI-O · The Final Step Before Your Application Starts

PART V

Making Your Application Reachable

11 Your Pods Are Running... But Nobody Can Reach Them

The Problem with Pod IPs · Why Pods Are Ephemeral · Why Kubernetes Needs Services

12 Services

What Is a Service? · ClusterIP · Virtual IPs · Service Selectors · Why Clients Never Talk Directly to Pods

13 EndpointSlices

Why EndpointSlices Exist · From Service Selector to Pod IP · How Kubernetes Knows Which Pods Belong to a Service · Why EndpointSlices Replaced Endpoints

14 kube-proxy

Where kube-proxy Fits Into the Story · Watching Services · Watching EndpointSlices · Programming iptables · IPVS · Packet Forwarding · Load Balancing Requests

15 Headless Services

What Changes When ClusterIP Is None? · DNS Instead of Virtual IPs · StatefulSets · Stable Network Identity · Why Databases Prefer Headless Services

16 Rolling Updates

What Happens When You Change the Image? · Creating a New ReplicaSet · Scaling Down the Old ReplicaSet · Scaling Up the New ReplicaSet · Zero-Downtime Deployments

17 Kubernetes Never Stops Watching

Controllers Never Sleep · The Watch API · Informers · Local Cache · Continuous Reconciliation · A Complete Walkthrough of the Reconciliation Loop

18 Trust Between Kubernetes Components

Why Security Matters · Mutual TLS · Public Keys · Private Keys · Certificate Authorities

19 Client and Server Certificates

Thinking Like Kubernetes · Who Connects to Me? · Who Do I Connect To? · API Server Certificates · etcd Certificates · Scheduler Certificates · Controller Manager Certificates · Kubelet Certificates · kube-proxy Certificates · kubect1 Certificates

20 kubeconfig

The Question kubeconfig Answers · Clusters · Users · Contexts · Current Context · How kubectl Finds the API Server · Mutual TLS in Practice

21 Following One Deployment From Start to Finish

A complete end-to-end walkthrough in 13 steps · the full lifecycle diagram

§ **Component Reference**

Every component: responsibility, what it watches, what it creates, common production issue

BM

Reference & Appendices

§ **Production Debugging Guide**

Where Should You Look First? · Pods Stuck in Pending · ImagePullBackOff · CrashLoopBackOff · Services Without Endpoints · Node NotReady · Scheduling Failures · DNS Failures · Networking Failures · Certificate Errors · Authentication Errors · Troubleshooting Flow

§ **Common Kubernetes Misconceptions**

Ten myths, corrected

§ **Final Thoughts**

Kubernetes Is Just a Collection of Specialized Controllers · Thinking in Desired State · Why This Mental Model Changes Everything · Where to Go Next

§ **Appendices A–E**

A · Complete Architecture Diagram · B · Certificate Relationships · C · Control Plane Components · D · Useful kubectl Commands · E · Further Reading

Preface

Don't memorize Kubernetes. Learn how it thinks.

Why I Wrote This Guide

The first time I learned Kubernetes, I found myself memorizing components rather than understanding how they worked together.

I knew that Deployments managed ReplicaSets, ReplicaSets managed Pods, the Scheduler assigned Pods to nodes, and the Kubelet started containers. I could define each component individually, but if someone had asked me a simple question such as, *"What actually happens after you run `kubectl apply`?"*, I wouldn't have been able to explain the complete journey.

Most tutorials teach Kubernetes by introducing one component at a time. While that approach explains what each component does, it rarely explains how they collaborate to transform a YAML file into a running application.

This guide takes a different approach.

Instead of studying Kubernetes as a collection of independent components, we'll follow a single Deployment from the moment it leaves your terminal until it becomes a running application inside the cluster. Along the way, you'll see every major Kubernetes component perform its role before handing responsibility to the next.

By the end of this guide, my goal isn't for you to memorize Kubernetes. It's for you to understand how it thinks. Once you understand that, troubleshooting becomes easier, architecture diagrams start making sense, and Kubernetes becomes much less intimidating.

Who This Guide Is For

This guide is written for engineers who want to move beyond writing Kubernetes YAML files and understand what happens behind the scenes. It is intended for:

- ◇ **Software Engineers** deploying applications to Kubernetes.
- ◇ **DevOps Engineers** managing Kubernetes environments.
- ◇ **Site Reliability Engineers** responsible for operating production clusters.
- ◇ **Platform Engineers** building internal developer platforms.
- ◇ **Students** preparing for Kubernetes certifications or interviews.
- ◇ **Anyone curious** about how modern container orchestration works.

You don't need to be an expert to follow along. If you've created a Deployment before or have a basic understanding of Pods and containers, you'll have everything you need.

Why Understanding Kubernetes Still Matters in the Age of AI

Today, AI can generate Kubernetes manifests, explain concepts, and even help troubleshoot cluster issues. That's incredibly useful, and it has changed the way many of us work. However, production systems don't fail in predictable ways.

When an application isn't starting, Pods remain in the Pending state, or a Service suddenly stops routing traffic, the most valuable skill isn't remembering commands. It's understanding how Kubernetes is making decisions.

- ◇ If you know that a Deployment doesn't create Pods directly, you'll investigate the **ReplicaSet** before looking elsewhere.

- ◇ If Pods are stuck in the Pending state, you'll know to investigate the **Scheduler** before checking the Kubelet.
- ◇ If a Service has no endpoints, you'll understand that the issue may have nothing to do with **kube-proxy**.

These aren't facts to memorize. They're a mental model for reasoning about the system. AI can accelerate your workflow, but it cannot replace an understanding of how distributed systems behave. The better you understand Kubernetes, the better you'll be at asking the right questions, interpreting the answers, and making informed decisions when something goes wrong. That understanding is what this guide aims to build.

How to Read This Guide

This guide is designed as a journey rather than a reference manual. We'll follow a single Deployment throughout the entire publication, using it to understand how Kubernetes components interact with one another. Each section builds on the previous one, so I recommend reading the guide in order, especially if you're new to Kubernetes internals.

Throughout the guide you'll also come across a few recurring callouts:

⚡ PRODUCTION INSIGHT

Lessons and patterns commonly encountered in real production environments.

🔍 BEHIND THE SCENES

What Kubernetes is doing internally beyond what you see from the command line.

⚠️ COMMON MISCONCEPTION

Ideas that often confuse engineers learning Kubernetes.

🔍 DEBUGGING TIP

How the concept can help you troubleshoot real-world issues.

✓ KEY TAKEAWAY

The most important point, summarized before moving on.

Whenever possible, follow along using your own Kubernetes cluster. Run the commands, inspect the resources, and observe how the cluster changes as we progress through the deployment lifecycle. Seeing these concepts in action makes them much easier to understand and remember.

Most importantly, don't focus on memorizing every component or command. Instead, focus on understanding the flow. **Once you understand the journey, the individual components naturally fall into place.**

The Journey Begins

One command, one Deployment, and the entire control plane springs into motion.

01 What Really Happens When You Run `kubectl apply` ?

The 2:17 AM Production Incident

It's 2:17 AM. Your phone vibrates with an alert from your monitoring system.

⚠ PRODUCTION ALERT

Deployment `payments-api` **has 0 of 3 Pods Ready.**

Customers are beginning to experience failed payment requests, and the incident response channel is already filling up with messages. You log in to the cluster and run the first command every Kubernetes engineer has typed at some point.

```
terminal
$ kubectl get pods
```

The output isn't what you expected.

```
output
```

NAME	READY	STATUS	RESTARTS	AGE
payments-api-7fd4bb9d8d-4v9mk	0/1	Pending	0	2m
payments-api-7fd4bb9d8d-8txnm	0/1	Pending	0	2m
payments-api-7fd4bb9d8d-rph17	0/1	Pending	0	2m

Three Pods exist. None of them are running. The application is unavailable.

Someone suggests deleting the Pods. Another engineer thinks restarting the nodes might fix the issue. At this point, neither action addresses the real problem. The first question an experienced Site Reliability Engineer asks is not:

"How do I restart Kubernetes?"

Instead, they ask:

"Where in the deployment lifecycle did Kubernetes stop?"

- Did the Deployment get created?
- Did the ReplicaSet create the Pods?
- Did the Scheduler assign those Pods to a worker node?
- Did the Kubelet receive the assignment?
- Or did the container fail after it started?

Notice how every one of those questions points to a different Kubernetes component. Understanding which component is responsible for each stage of the deployment lifecycle turns debugging from guesswork into reasoning. That is exactly what this guide is about.

The Problem with Learning Kubernetes Component by Component

Open almost any Kubernetes tutorial and you'll see chapters like these:

API Server

etcd

Scheduler

ReplicaSet

Kubelet

kube-proxy

While each chapter explains the purpose of a component, they rarely explain how these components work together. As a result, many engineers understand Kubernetes as a list of services rather than a distributed system. For example, you might know that:

- ◇ Deployments manage ReplicaSets.
- ◇ ReplicaSets manage Pods.
- ◇ The Scheduler assigns Pods to nodes.
- ◇ The Kubelet starts containers.

Those statements are all correct. But they don't answer a more important question: **what actually happens after I run `kubectl apply` ?** Understanding Kubernetes isn't about memorizing responsibilities. It's about understanding how responsibility moves from one component to another until your application is running. Once you understand that flow, Kubernetes becomes significantly easier to reason about.

A Different Way to Learn Kubernetes

Rather than studying Kubernetes one component at a time, we're going to follow a single Deployment from beginning to end. We'll use the following Deployment throughout this guide.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments-api
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payments-api
  template:
    metadata:
      labels:
        app: payments-api
    spec:
      containers:
      - name: payments-api
        image: company/payments:v1
        ports:
        - containerPort: 8080
```

This Deployment is the only object we'll create ourselves. Everything else you'll see in this guide exists because Kubernetes reacts to this Deployment.

- ◇ The Deployment becomes a **ReplicaSet**.
- ◇ The ReplicaSet creates **Pods**.
- ◇ The Scheduler assigns Pods to **worker nodes**.
- ◇ The Kubelet starts **containers**.
- ◇ A **Service** makes the application discoverable.
- ◇ **EndpointSlices** keep track of the available Pods.
- ◇ **kube-proxy** forwards traffic.

Every chapter represents another step in that journey.

Our Journey Through the Control Plane

The deployment lifecycle we'll follow looks like this:

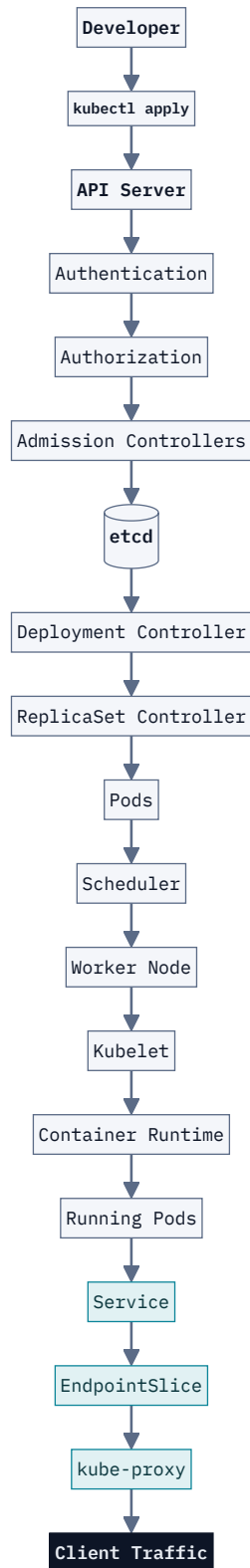


Figure 1.1 – End-to-End Deployment Lifecycle

Don't worry if some of these components are unfamiliar. By the end of this guide, you'll understand why every one of them exists and how they work together.

Desired State: The One Concept Everything Builds Upon

Before we begin exploring the control plane, there is one idea you need to understand. Everything in Kubernetes revolves around **desired state**.

When you create a Deployment, you are not giving Kubernetes a list of instructions. You are describing the outcome you want. In our example, we're saying:

"I want three healthy instances of the payments-api application running."

Notice what we didn't specify:

- ◇ We didn't choose which worker node should run the Pods.
- ◇ We didn't tell Kubernetes how to recover if one of the Pods crashes.
- ◇ We didn't define which order containers should start.

We simply described the desired outcome. From that moment onward, Kubernetes continuously compares the current state of the cluster with the desired state you've declared. Whenever the two differ, Kubernetes takes action to bring them back into alignment.

This process is known as **reconciliation**, and it is the fundamental principle behind every controller you'll encounter throughout this guide. It's also the reason Kubernetes is often described as a declarative orchestration platform rather than an imperative one.

You declare what you want. Kubernetes decides how to achieve it.

✧ PRODUCTION INSIGHT

One of the biggest mindset shifts for engineers coming from traditional infrastructure is learning to stop managing individual servers and start managing desired outcomes. In Kubernetes, you don't restart failed Pods because you expect them to fail — you describe the state you want, and Kubernetes continuously works to maintain it.

This declarative model is one of the key reasons Kubernetes can automate recovery, perform rolling updates, and scale applications without requiring manual intervention.

✓ KEY TAKEAWAY

Kubernetes is not a collection of independent components. It is a **system of specialized controllers working together to transform a desired state into reality**. Every chapter that follows represents another step in that journey.

Next, we'll press Enter on our `kubectl apply` command and watch the first component of the control plane receive our request: the Kubernetes API Server.

Entering the Cluster

Before Kubernetes can act on your intent, it has to receive it, check it, and remember it.

"Every request entering a Kubernetes cluster begins here."

02 The Kubernetes API Server: The Front Door to the Cluster

In the previous chapter, we introduced our `payments-api` Deployment and learned that Kubernetes operates by continuously working toward a desired state. Now it's time to submit that desired state to the cluster.

We run the following command:

```
$ kubectl apply -f deployment.yaml
```

From the terminal, the command finishes in a fraction of a second. Behind the scenes, however, Kubernetes has only just begun its work. The first stop is the Kubernetes API Server.

Why Every Request Starts Here

One of the first things you'll notice when learning Kubernetes is that almost every component communicates with the API Server. Whether you're:

creating a Deployment

updating a Service

deleting a Pod

viewing cluster resources

scheduling workloads

watching for changes

everything begins with the API Server. It is the entry point into the Kubernetes control plane. More importantly, it is the **only component allowed to modify the cluster's state**. That means even Kubernetes components themselves don't bypass the API Server.

- ◇ The Scheduler doesn't communicate directly with etcd.
- ◇ The Deployment Controller doesn't create ReplicaSets by writing into etcd.
- ◇ The Kubelet doesn't update Pod status directly.

Every component communicates through the API Server. This design gives Kubernetes a single source of truth for every change happening inside the cluster.

⚙️ PRODUCTION INSIGHT

One of the easiest ways to think about the API Server is as the receptionist of a busy office building. Nobody walks directly into individual offices — every visitor checks in at reception first. Likewise, every request entering a Kubernetes cluster goes through the API Server before reaching any other component.

Receiving the Request

Let's go back to our Deployment.

```
terminal
$ kubectl apply -f deployment.yaml
```

Although it feels like you're talking directly to Kubernetes, that's not exactly what's happening. The `kubectl` command is simply a client. It reads your kubeconfig file to determine:

- ◇ Which cluster to connect to.
- ◇ The API Server's address.
- ◇ How to authenticate.

◇ Which certificate authority to trust.

Once that information has been loaded, `kubectl` sends an HTTPS request to the API Server.

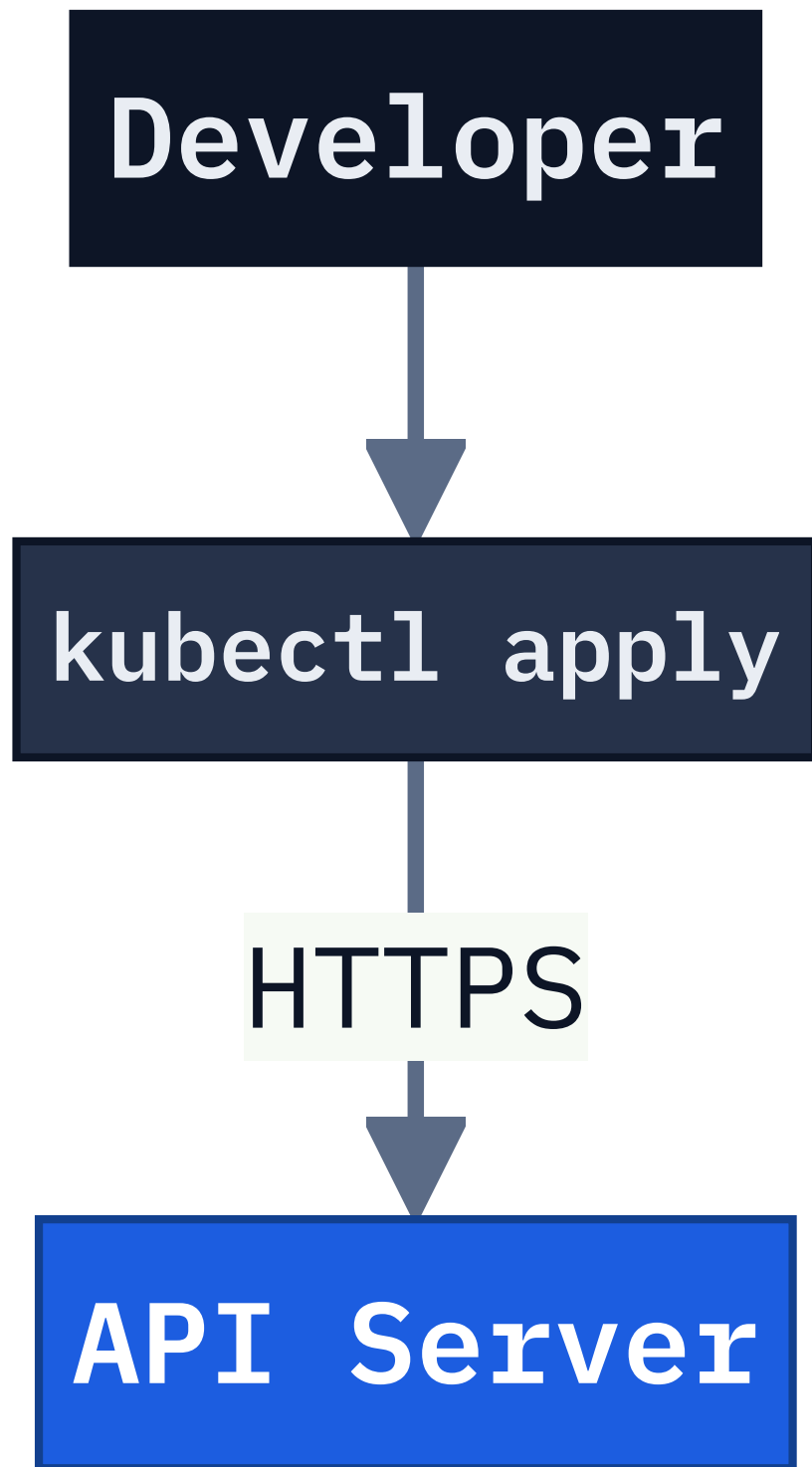


Figure 2.1 – Submitting a Deployment

At this point, the API Server has received your Deployment, but it has not accepted it yet. Before it stores anything, it must first answer two important questions:

Authentication

Authentication answers the first question: **who is making this request?** The API Server doesn't automatically trust every incoming request. Instead, it verifies the identity of the client before doing anything else. Depending on how the cluster is configured, authentication may be performed using:

Client certificates

Bearer tokens

OpenID Connect (OIDC)

Cloud provider identities (IAM)

If authentication fails, the request ends immediately. The API Server never reaches the Deployment manifest because it cannot verify who submitted it.

Example

Suppose someone attempts to create a Deployment without valid credentials.

```
$ kubectl apply -f deployment.yaml
```

The API Server responds:

RESPONSE

Error from server (Unauthorized): Unauthorized

No Deployment is created. Nothing is written to etcd. The request ends here.

Authorization

Authentication tells Kubernetes *who you are*. Authorization determines *what you're allowed to do*. Imagine two engineers connecting to the same cluster. Alice is responsible for the development namespace. Bob manages production. Both successfully authenticate.

However, when Alice attempts to deploy an application into the production namespace, Kubernetes checks whether her identity has permission to perform that action. If not, the API Server rejects the request. A typical response looks like this:

RESPONSE

```
Error from server (Forbidden): User "alice" cannot create deployments in
namespace "production"
```

Notice the difference. Authentication succeeded. Authorization failed. Understanding this distinction is important because they solve two completely different problems.

AUTHENTICATION	AUTHORIZATION
Who are you?	What are you allowed to do?

Admission Controllers

If authentication and authorization both succeed, the API Server performs one final series of checks before accepting the request. These checks are handled by **Admission Controllers**. Think of Admission Controllers as the cluster's quality gate. Their job is to inspect every incoming request before it becomes part of the cluster's desired state. Some Admission Controllers modify the object. Others validate it. If validation fails, the request is rejected.

Mutating vs Validating Admission Controllers

Admission Controllers fall into two categories.

Mutating Admission Controllers

These are allowed to modify the object before it is stored. For example, an Admission Controller may:

- ◆ Add default labels.
- ◆ Inject a service mesh sidecar.
- ◆ Apply default resource limits.
- ◆ Attach additional annotations.

The object you submitted isn't necessarily the object that gets stored.

Validating Admission Controllers

Validation happens after all mutations are complete. Their responsibility is simple: reject anything that violates the cluster's rules. For example:

- ◆ Missing required labels.
- ◆ Invalid image names.
- ◆ Security policy violations.
- ◆ Resource quota limits.

If validation fails, Kubernetes rejects the request. Nothing is stored.

Why Everything Goes Through the API Server

At this point, you might wonder why controllers don't simply communicate directly with each other. Why doesn't the Scheduler create Pods? Why doesn't

the Deployment Controller write directly into etcd? The answer is **consistency**. Every modification to the cluster passes through a single component. That means Kubernetes has:

- ◇ one authentication mechanism
- ◇ one authorization mechanism
- ◇ one validation pipeline
- ◇ one audit trail
- ◇ one source of truth

Without the API Server acting as the central coordinator, different components could make conflicting changes, bypass security policies, or overwrite each other's updates. Centralizing all write operations ensures that every change follows the same rules and that the cluster remains consistent.

Why Components Don't Talk Directly to Each Other

This is one of Kubernetes' most elegant design decisions. Controllers don't need to know about each other. Instead, they communicate indirectly by observing the state stored through the API Server. For example:

- ◇ The Deployment Controller creates a ReplicaSet by sending a request to the API Server.
- ◇ The ReplicaSet Controller notices the new ReplicaSet by watching the API Server.
- ◇ The Scheduler notices newly created Pods by watching the API Server.
- ◇ The Kubelet notices Pods assigned to its node by watching the API Server.

Every component reacts to changes in the cluster's state rather than calling another component directly. This loosely coupled design makes Kubernetes easier to scale, easier to extend, and more resilient to failures.

✓ KEY TAKEAWAY

The API Server is much more than a REST API — it is the **communication hub of the Kubernetes control plane**. Every request, whether from a user or another Kubernetes component, passes through it. Before accepting any change, the API Server:

1. Authenticates the client.
2. Authorizes the request.
3. Runs Admission Controllers.
4. Persists the object to **etcd**, the cluster's datastore.

Once those steps are complete, the Deployment has officially entered the cluster. But something interesting happens next: nothing is running. No Pods exist. No containers have started. The Deployment is simply stored as data.

*So where does Kubernetes keep that data? In the next chapter, we'll explore **etcd** — the distributed key-value store that acts as Kubernetes' source of truth.*

"The API Server accepted my Deployment. Where did it go?"

03 etcd: Kubernetes' Source of Truth

In the previous chapter, our Deployment successfully passed authentication, authorization, and admission control. The API Server accepted the request — but something interesting happened.

Nothing started running. No Pods were created. No containers were launched. No worker node received any instructions. At this point, our Deployment exists only as a Kubernetes object. So where is it stored? The answer is **etcd**.

What Is etcd?

etcd is a distributed key-value database used by Kubernetes to store the state of the cluster. Think of it as Kubernetes' memory. Whenever you create, update, or delete a Kubernetes object, the API Server persists that change in etcd. Whether it's a Deployment, Pod, Service, ConfigMap, Secret, or Namespace, every object ultimately lives in etcd.

If the API Server is the front door to the cluster, then etcd is the vault where Kubernetes keeps everything it knows about itself.

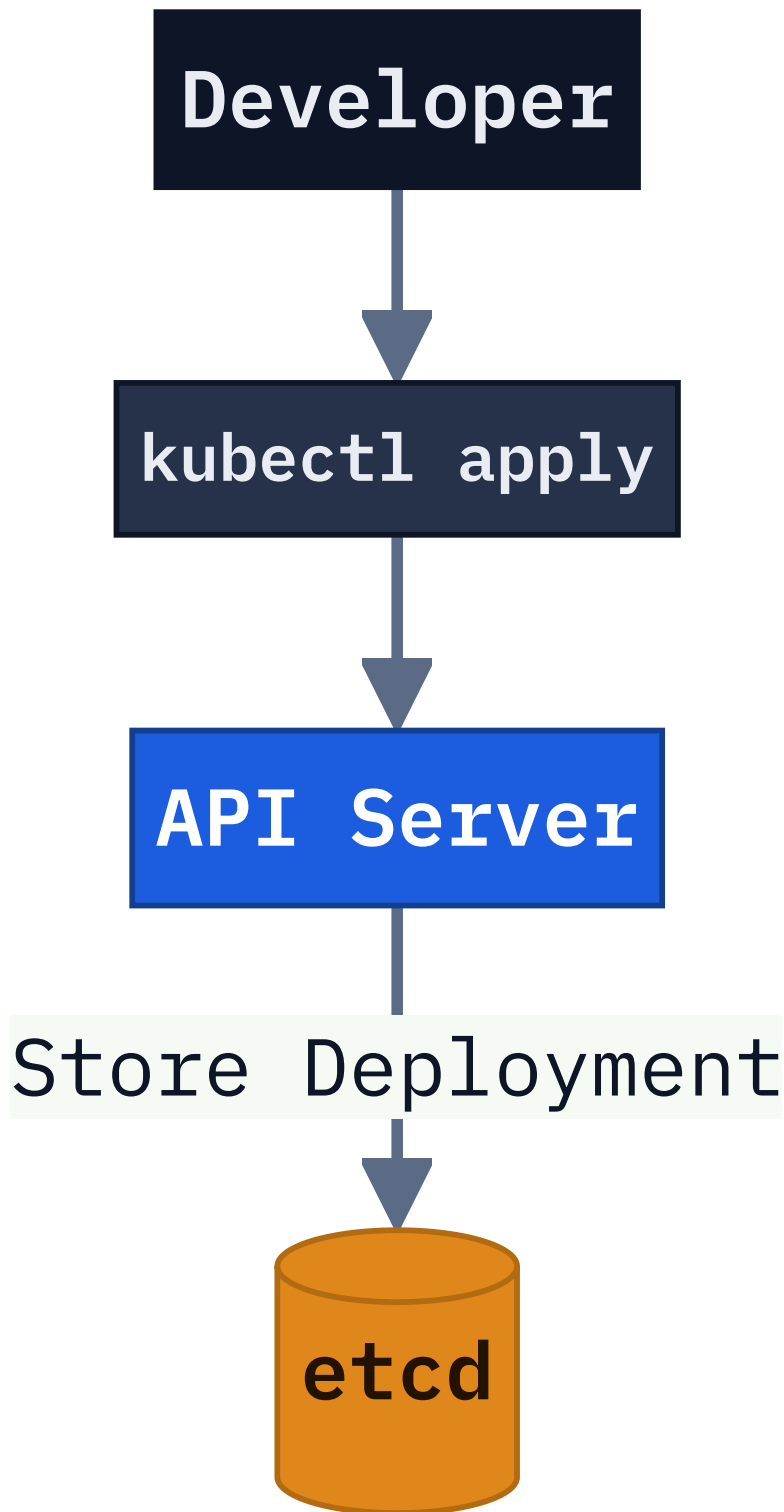


Figure 3.1 – The API Server Persists Objects in etcd

Notice that the API Server is the only component writing to etcd. This is an intentional design decision. By ensuring that every change passes through

the API Server, Kubernetes maintains a single, secure path for modifying the cluster's state.

Why Kubernetes Needs a Distributed Key-Value Store

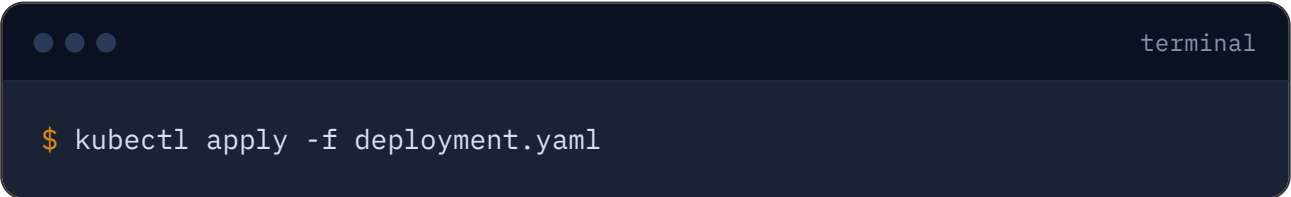
Imagine managing thousands of applications across hundreds of worker nodes. Kubernetes needs a reliable way to answer questions such as:

- ◇ Which Deployments exist?
 - ◇ How many replicas should each Deployment have?
 - ◇ Which Pods belong to which ReplicaSet?
 - ◇ Which Services exist?
 - ◇ Which Pods are currently running?
 - ◇ Which node is each Pod assigned to?
-

Without a central source of truth, every component would maintain its own copy of the cluster's state, leading to inconsistencies and synchronization problems. Instead, Kubernetes stores everything in one place. Every component reads from the same source of truth. Every component writes through the API Server. This allows the entire control plane to remain consistent.

What Happens After Your Deployment Is Accepted?

Let's return to our example.



```
$ kubectl apply -f deployment.yaml
```

A terminal window with a dark background. The title bar shows three window control buttons on the left and the word 'terminal' on the right. The command prompt is a yellow dollar sign followed by the command 'kubectl apply -f deployment.yaml'.

After the API Server finishes validating the request, it persists a new Deployment object to etcd. Conceptually, that object looks something like this.

```
Deployment object · conceptual

Deployment
-----
Name:      payments-api
Namespace: production
Replicas:  3
Image:     company/payments:v1
Labels:
  app: payments-api
Selector:
  app: payments-api
```

This is not the actual representation stored in etcd, but it illustrates the information Kubernetes has recorded about your Deployment. Notice something important: **Kubernetes has stored what you want, not what currently exists.** There are still:

◆ No ReplicaSets

◆ No Pods

◆ No scheduled workloads

◆ No running containers

The Deployment has been recorded, but Kubernetes hasn't acted on it yet.

What Actually Gets Stored?

⚠ COMMON MISCONCEPTION

One of the most common misconceptions is that etcd stores running containers. **It doesn't.** etcd stores Kubernetes objects — the containers themselves run on worker nodes.

For example, when you create a Deployment, etcd stores the Deployment object. Later, when the Deployment Controller creates a ReplicaSet, that ReplicaSet is also stored in etcd. When the ReplicaSet creates Pods, those Pod objects are stored as well. At every stage, Kubernetes is storing descriptions of the desired and observed state, not the containers themselves.

The containers run on worker nodes. The objects describing them live in etcd. This distinction is important because Kubernetes manages infrastructure through objects, not through direct interaction with containers.

Desired State vs Current State

To understand why etcd is so important, we need to revisit the idea of desired state. When we created our Deployment, we declared that we wanted three replicas of the `payments-api` application. That desired state is now stored in etcd. However, the current state of the cluster still looks like this:

RESOURCE	CURRENT STATE
Deployment	✓ Exists
ReplicaSet	✗ Does not exist
Pods	✗ Do not exist
Containers	✗ Not running

This difference between the desired state and the current state is what drives Kubernetes. The cluster now knows what you want, but reality hasn't caught up yet. The next step is for Kubernetes to notice that difference and begin reconciling it.

Why Nothing Is Running Yet

This is one of the most surprising moments in the deployment lifecycle. Many engineers assume that the API Server starts Pods. **It doesn't.** The API Server's responsibility ends after storing the Deployment. Its job is to receive requests, validate them, and persist them.

- ◇ It never schedules Pods.
- ◇ It never creates containers.
- ◇ It never chooses worker nodes.

Instead, Kubernetes relies on another group of components that are continuously watching the API Server for changes. These components are called **controllers**. The moment our Deployment was stored in etcd, the Deployment Controller became interested. It noticed that a new Deployment had appeared. What happens next is where Kubernetes truly comes to life.

PRODUCTION INSIGHT

One of the first places experienced Kubernetes engineers look during troubleshooting is the API Server and etcd. If a Deployment never appears after running `kubectl apply`, the problem likely occurred *before* the object was stored — an authentication issue, an authorization failure, or a validation error.

If the Deployment exists but no ReplicaSet has been created, the problem lies further along the control plane. At that point the API Server has already completed its work, and attention should shift to the controllers responsible for reconciliation.

✓ KEY TAKEAWAY

etcd is Kubernetes' source of truth. It doesn't run containers or schedule workloads — it stores the objects that describe the **desired and current state** of the cluster. When we submitted our Deployment, Kubernetes recorded our intent but didn't immediately act on it. That responsibility belongs to the controllers.

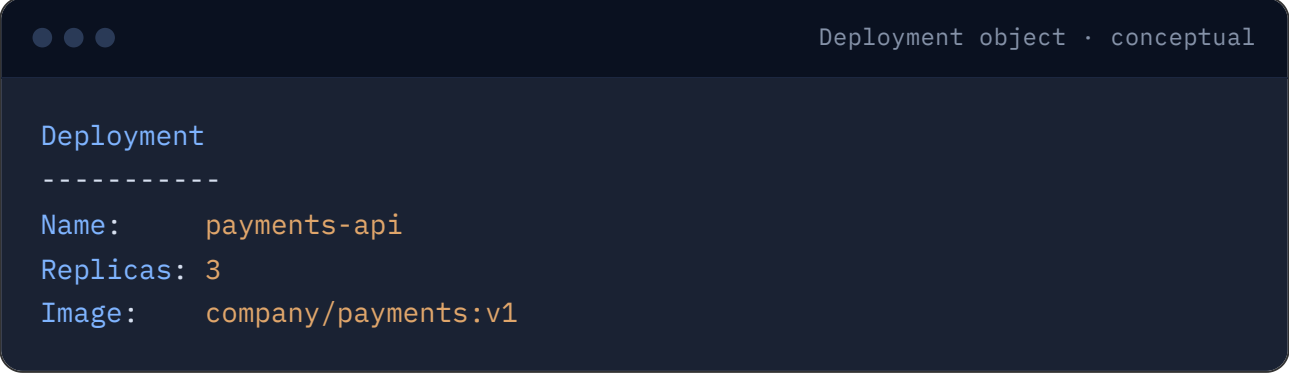
In the next chapter, we'll meet the components that give Kubernetes its self-healing capabilities. We'll explore what controllers are, why they exist, and how they continuously reconcile the actual state of the cluster with the desired state stored in etcd.

Kubernetes Starts Working

Nothing in Kubernetes happens because one component tells another what to do — everything happens because components observe the cluster's state and reconcile it.

04 Understanding Controllers: The Reconciliation Loop That Powers Kubernetes

In the previous chapter, our Deployment was successfully stored in etcd. At this point, Kubernetes knows exactly what we want.

A terminal window with a dark background. The title bar shows three window control buttons on the left and the text "Deployment object · conceptual" on the right. The terminal content shows the word "Deployment" followed by a dashed line separator. Below the separator, the following text is displayed: "Name: payments-api", "Replicas: 3", and "Image: company/payments:v1".

```
Deployment
-----
Name:      payments-api
Replicas:  3
Image:     company/payments:v1
```

Yet if you were to inspect the cluster right now, you would notice something surprising. There are still no ReplicaSets, no Pods, and no running containers. The Deployment exists only as data. So what happens next? The answer lies in one of Kubernetes' most important concepts: **controllers**.

Why They're Called Controllers

The word *controller* comes from control theory, a branch of engineering concerned with keeping systems operating at a desired state despite changes in their environment. A simple example is the thermostat in your home.

Suppose you set the desired temperature to 22°C. The thermostat continuously measures the current temperature. If the room cools to 20°C, it turns on the heater. Once the room reaches 22°C, it turns the heater off.

The thermostat isn't running on a timer. It isn't asking someone whether it should turn the heater on. It continuously compares reality with the desired

state and acts only when the two don't match. Kubernetes controllers work in exactly the same way – instead of controlling temperature, they control Kubernetes resources. Instead of asking *"Did someone tell me to create a Pod?"* they ask *"Does the current state of the cluster match the desired state?"* If the answer is no, they take action.

Desired State vs Actual State

Let's go back to our Deployment. The desired state stored in etcd says

`Replicas = 3`, but the actual state of the cluster looks like this.

RESOURCE	CURRENT STATE
Deployment	✓ Exists
ReplicaSet	✗ Missing
Pods	✗ Missing

The cluster knows what should exist. Reality doesn't match. That difference is called **drift**. The entire purpose of Kubernetes is to continuously eliminate that drift. Every controller in Kubernetes exists for one reason: **bring the actual state closer to the desired state**.

The Reconciliation Loop

Controllers don't perform an action once and stop. Instead, they continuously repeat the same process:

- ◇ Observe the current state.
 - ◇ Compare it with the desired state.
 - ◇ Take action if they differ.
-

This process is known as the **reconciliation loop**. It's the heartbeat of Kubernetes, and every major controller follows this pattern.

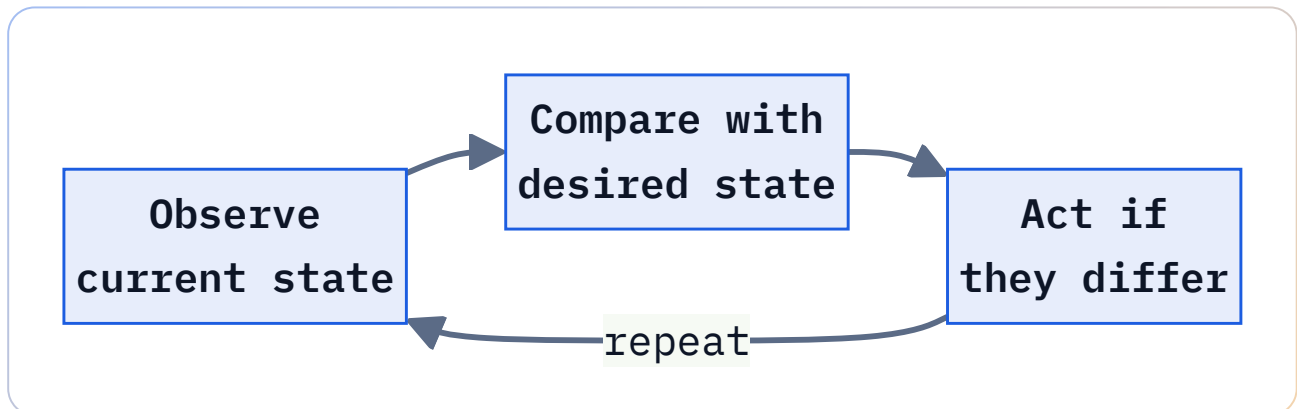


Figure 4.1 – The Reconciliation Loop

For example, imagine a Deployment requesting three replicas. Initially, the cluster looks like this: `Desired Pods: 3` · `Current Pods: 0`. The controller detects the difference and begins reconciling it. Later, one Pod crashes, and the cluster now looks like this: `Desired Pods: 3` · `Current Pods: 2`. Once again, the controller notices the difference and creates another Pod.

Notice something important. Nobody tells the controller that a Pod crashed. It simply observes the cluster's current state and reacts. This is why Kubernetes is described as **self-healing**.

Event-Driven Architecture

At this point, you might wonder: does Kubernetes constantly check every object in the cluster? Not exactly. That would be inefficient, especially in clusters running thousands of workloads. Instead, Kubernetes follows an **event-driven architecture**. Whenever something changes, the API Server publishes an event. Controllers listen for those events and respond only when something relevant changes.

Instead of repeatedly asking *"Has anything changed yet?"* the controller simply waits until the API Server tells it that something has changed. This makes

Kubernetes highly scalable.

The Watch API

So how do controllers receive those updates? Through the **Watch API**. Rather than repeatedly sending requests every few seconds, a controller opens a long-lived HTTP connection to the API Server. That connection remains open. Whenever an object is created, updated, or deleted, the API Server immediately streams an event to the controller.

Think of it like subscribing to live notifications instead of constantly refreshing a webpage. For example, when we created our Deployment, the API Server immediately notified the Deployment Controller that a new Deployment object had been added. That notification is what triggers the next stage of our deployment journey.

Informers

If every controller queried the API Server directly for every event, the API Server would quickly become overwhelmed. To solve this problem, Kubernetes uses **Informers**. An Informer watches the API Server on behalf of a controller and maintains a local copy of the resources it cares about. Whenever something changes, the Informer updates its local data and notifies the controller. This means the controller rarely needs to query the API Server directly — instead, it works from a fast, in-memory cache.

Local Cache

You can think of the Informer's cache as the controller's notebook. Instead of asking the API Server the same questions repeatedly, the controller keeps a synchronized copy of the information it needs. When the API Server reports a change, only the affected objects are updated. This approach dramatically

reduces the number of requests made to the API Server while still keeping controllers informed of changes almost immediately. It's one of the reasons Kubernetes can scale to very large clusters.

Why Kubernetes Doesn't Continuously Poll

Polling would mean repeatedly asking the API Server questions such as *"Has a new Deployment been created?"*, *"Has a Pod disappeared?"*, or *"Has a Service changed?"* Even if nothing changed, every request would still consume resources. Instead, Kubernetes uses watches. Controllers remain idle until the API Server notifies them of a relevant event. This event-driven approach is significantly more efficient than polling and allows Kubernetes to react quickly while minimizing unnecessary work.

Production Example

Let's return to our `payments-api` Deployment. When we executed `kubectl apply -f deployment.yaml`, the API Server persisted the Deployment to etcd. Immediately afterward:

- ◇ The Deployment Controller received a notification that a new Deployment had been created.
 - ◇ It compared the desired state with the current state.
 - ◇ It noticed that no ReplicaSet existed.
 - ◇ It began the reconciliation process.
-

At this point, the Deployment Controller becomes the next component responsible for our application's journey. We'll explore exactly what it does in the next chapter.

⚠ COMMON MISCONCEPTION

Controllers constantly poll the API Server. Not quite. Controllers use the Watch API together with Informers to receive change notifications and maintain a synchronized local cache. This event-driven model allows Kubernetes to react quickly without repeatedly querying the API Server.

💡 PRODUCTION INSIGHT

Once you understand the reconciliation loop, many Kubernetes behaviors begin to make sense. Deleting a Pod doesn't permanently remove it, because the ReplicaSet Controller notices that the actual number of Pods no longer matches the desired number.

Similarly, manually editing a managed resource may not have the effect you expect. If your change conflicts with the desired state stored in the cluster, the appropriate controller will eventually reconcile it back. Rather than thinking of Kubernetes as executing commands, think of it as continuously correcting drift.

✓ KEY TAKEAWAY

Controllers are the **decision-makers of Kubernetes**. They don't communicate directly with one another, and they don't wait for manual instructions. Instead, they observe the cluster through the API Server, compare the desired state with the actual state, and continuously reconcile any differences.

Our Deployment has now caught the attention of the Deployment Controller. It has noticed that a Deployment exists without a corresponding ReplicaSet. Its next job is to create the first ReplicaSet and move our application one step closer to running.

"From a Deployment to a ReplicaSet."

05 The Deployment Controller

At the end of the previous chapter, our Deployment had successfully entered the cluster. The API Server had authenticated the request, authorized it, validated it through the admission controllers, and stored it in etcd. If we query the cluster now, we'll find our Deployment.

```
terminal
$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
payments-api	0/3	0	0	5s

Notice something interesting. The Deployment already exists, but no Pods are running yet. The API Server has finished its work. The next step belongs to another Kubernetes component: the **Deployment Controller**.

Discovering a New Deployment

Earlier, we learned that controllers continuously watch the API Server using the Watch API. As soon as our Deployment was stored in etcd, the Deployment Controller received a notification that a new Deployment object had been created. It didn't receive a direct message from the API Server saying *"Please create a ReplicaSet."* Instead, it simply observed that the cluster's current state didn't match the desired state. Our Deployment requested three replicas, but no ReplicaSet existed to manage them. From the controller's perspective, the cluster looked like this:

RESOURCE	CURRENT STATE
Deployment	✓ Exists
ReplicaSet	× Missing
Pods	× Missing

This difference is enough to trigger another reconciliation cycle.

The Deployment Controller's Responsibility

One of the biggest misconceptions about Kubernetes is that a Deployment creates Pods. It doesn't. The Deployment Controller has only one primary responsibility: **create and manage ReplicaSets**. That's it. It never schedules Pods. It never creates containers. It never chooses worker nodes. Instead, it creates a ReplicaSet that will eventually become responsible for managing Pods.

Think of the Deployment Controller as a project manager. It doesn't build the application itself — instead, it creates another resource that knows how to manage replicas.

Creating the First ReplicaSet

The Deployment Controller now creates a ReplicaSet. Notice something important. Just like every other Kubernetes component, it doesn't write directly to etcd. Instead, it sends another request to the API Server. The sequence now looks like this.

```
graph TD; DC[Deployment Controller] --> CS[Create ReplicaSet]; CS --> AS[API Server]; AS --> etcd[(etcd)]
```

Deployment Controller

Create ReplicaSet

API Server

etcd

Figure 5.1 – Creating the First ReplicaSet

The API Server validates the request, stores the ReplicaSet object in etcd, and immediately publishes another event. The Deployment Controller's work is now complete.

What Does the ReplicaSet Look Like?

If we inspect the cluster again, we'll notice something new.


```
terminal

$ kubectl get replicaset

NAME                                DESIRED  CURRENT  READY
payments-api-6db7df6c9             3        0        0
```

A ReplicaSet now exists. However, look carefully: `CURRENT = 0` and `READY = 0`. There are still no running Pods. The ReplicaSet exists only as another Kubernetes object stored in etcd.

Why Deployments Never Create Pods Directly

This design often surprises engineers who are new to Kubernetes. Why introduce another resource? Why not let Deployments create Pods directly? The answer is **separation of responsibilities**. A Deployment is responsible for managing application updates. A ReplicaSet is responsible for maintaining the desired number of Pods. Keeping these responsibilities separate allows Kubernetes to perform rolling updates without losing track of existing Pods.

Suppose you later update your Deployment.

```
payments-api.yaml

image: company/payments:v2
```

The Deployment Controller won't modify the existing ReplicaSet. Instead, it creates an entirely new ReplicaSet. The old ReplicaSet gradually scales down while the new ReplicaSet gradually scales up. We'll explore this process in detail when we discuss rolling updates.

Handing Responsibility to the ReplicaSet

At this point, the Deployment Controller has completed its work. The cluster now looks like this.

RESOURCE	CURRENT STATE
Deployment	✓ Exists
ReplicaSet	✓ Exists
Pods	✗ Missing

Just like before, another controller notices the difference. The ReplicaSet Controller observes that the desired number of Pods is three, but none currently exist. Its responsibility is clear: **create the missing Pods**. Our Deployment has now moved one step further along its journey.

▶ TRY IT YOURSELF

You can observe these resources appear in sequence in your own cluster. Run these commands quickly after creating the Deployment — watching the resources appear one after another is one of the best ways to understand how Kubernetes reacts to changes.

```
terminal
$ kubectl apply -f deployment.yaml # create the Deployment
$ kubectl get deployment           # inspect the Deployment
$ kubectl get rs                  # inspect the ReplicaSet
$ kubectl get pods                # inspect the Pods
```

✦ PRODUCTION INSIGHT

Because Deployments own ReplicaSets rather than Pods, deleting a Pod rarely solves deployment issues. If a Pod disappears, the ReplicaSet simply creates another one to maintain the desired number of replicas.

Similarly, editing a Pod that's managed by a Deployment is usually temporary — the next reconciliation cycle may replace it if the changes don't match the Deployment's desired state. This is why production engineers typically modify the Deployment rather than individual Pods.

⚠ COMMON MISCONCEPTION

Deployments create Pods. Not quite. The Deployment Controller creates a ReplicaSet; the ReplicaSet Controller creates Pod objects. Keeping these responsibilities separate allows Kubernetes to perform rolling updates, rollbacks, and self-healing without coupling application lifecycle management to Pod management.

✓ KEY TAKEAWAY

The Deployment Controller doesn't run applications, schedule workloads, or create containers. Its responsibility is to ensure that every Deployment has a corresponding **ReplicaSet** capable of managing the desired number of Pods. Once the ReplicaSet has been created, the Deployment Controller steps aside.

The next stage of our journey belongs to the ReplicaSet Controller, whose job is to create the Pods that will eventually become our running application.

"Keeping your application alive."

06 ReplicaSets: Keeping Your Application Alive

In the previous chapter, the Deployment Controller detected our new Deployment and created a ReplicaSet. If we inspect the cluster, we'll see something like this:

```
$ kubectl get rs
```

NAME	DESIRED	CURRENT	READY
payments-api-6db7df6c9	3	0	0

The ReplicaSet now exists. But notice something interesting. Although it knows we want three replicas, there are still no Pods running. The Deployment Controller has already completed its work. Now it's the ReplicaSet Controller's turn.

The Desired Replica Count

The ReplicaSet's primary responsibility is simple: **ensure that the number of Pods running in the cluster always matches the desired number of replicas.** In our Deployment, we requested three replicas.

```
spec:
  replicas: 3
```

That value is passed to the ReplicaSet. The ReplicaSet immediately compares two values.

DESIRED STATE	ACTUAL STATE
3 Pods	0 Pods

The difference is obvious. Three Pods should exist. None currently do. This difference triggers another reconciliation cycle. Unlike the Deployment Controller, the ReplicaSet Controller doesn't create another Kubernetes resource — its responsibility is to create **Pod objects**.

Creating Pod Objects

As with every Kubernetes component we've encountered so far, the ReplicaSet Controller never writes directly to etcd. Instead, it sends a request to the API Server requesting the creation of three Pod objects. The sequence now looks like this:

ReplicaSet Controller

Create Pod Objects

API Server

etcd

Figure 6.1 – The ReplicaSet Controller Creates Pod Objects

Once the API Server validates the request, it stores three new Pod objects in etcd. If we inspect the cluster again with `kubectl get pods`, we'll see something similar to this:

output				
NAME	READY	STATUS	RESTARTS	AGE
payments-api-6db7df6c9-2fmlk	0/1	Pending	0	3s
payments-api-6db7df6c9-gm9qt	0/1	Pending	0	3s
payments-api-6db7df6c9-xb2pr	0/1	Pending	0	3s

This often surprises people. The Pods now exist. But they're still not running.

Why Pods Aren't Running Yet

⚠️ COMMON MISCONCEPTION

One of the biggest misconceptions in Kubernetes is believing that **creating a Pod automatically starts a container**. That's not what happens. At this stage, Kubernetes has only created Pod objects — descriptions of what should run — that haven't been assigned to a worker node yet.

If we inspect one of the Pods:

```
terminal
$ kubectl describe pod payments-api-6db7df6c9-2fmlk
```

we'll notice something important: the Pod has not yet been assigned to any node. That decision belongs to the Kubernetes Scheduler, the next component in our deployment journey.

What Happens When a Pod Dies? — Continuous Reconciliation

The same loop that creates Pods also keeps them alive. If a running Pod later disappears, the ReplicaSet notices the drift between desired and current

counts and creates a replacement — no human intervention required.

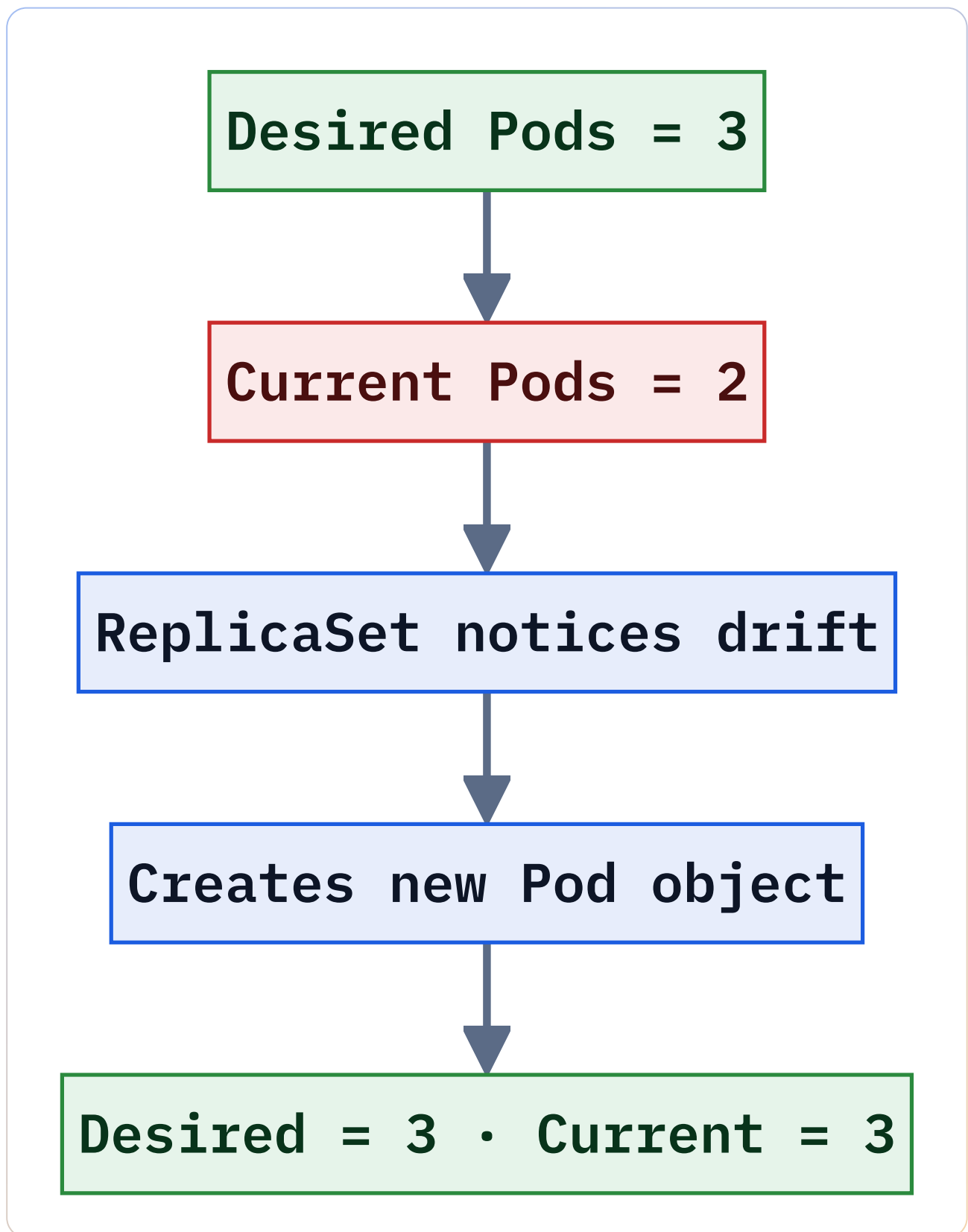


Figure 6.2 – Self-Healing: the ReplicaSet Corrects Drift

This is the reconciliation loop in action, and it's exactly why Kubernetes is described as self-healing: the ReplicaSet never stops comparing the desired replica count with reality.

✓ KEY TAKEAWAY

The ReplicaSet Controller has one job: keep the number of Pods matching the desired replica count. It creates **Pod objects** — not running containers. Those Pods exist in etcd, unscheduled and Pending, until another component decides where they should run.

Our Pods now exist, but they have nowhere to run. In the next chapter, we'll follow them as they wait for a home — and meet the component that decides where each one lands: the Kubernetes Scheduler.

Scheduling Workloads

The Pods exist — but they have nowhere to run. Now Kubernetes has to decide where each one lands, and then bring it to life.

"A Pod cannot run until Kubernetes decides where it should run."

07 Pods Waiting for a Home: Understanding the Pending State

At the end of the previous chapter, the ReplicaSet Controller had successfully created three Pod objects. If we inspect the cluster, we'll see them immediately.

```
terminal
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
payments-api-6db7df6c9-2fmlk	0/1	Pending	0	5s
payments-api-6db7df6c9-gm9qt	0/1	Pending	0	5s
payments-api-6db7df6c9-xb2pr	0/1	Pending	0	5s

This is one of the first moments that confuses many Kubernetes engineers. The Pods exist. The ReplicaSet has finished its work. So why aren't the containers running? The answer is simple: **a Pod cannot run until Kubernetes decides where it should run.** At this stage, the Pods exist only as Kubernetes objects stored in etcd. They haven't been assigned to any worker node.

Pending Pods

The Pending phase doesn't mean the Pod has failed. It simply means Kubernetes hasn't finished preparing it to run. A Pod enters the Pending state immediately after it's created. Before it can become a running application, Kubernetes still needs to:

- ◇ Select a worker node.
- ◇ Reserve resources on that node.
- ◇ Pull the container image.
- ◇ Configure networking.
- ◇ Mount any required volumes.
- ◇ Start the container.

Only after these steps are complete does the Pod transition to the Running state.

Why **Node = None**

Let's inspect one of the Pods.

```
terminal
$ kubectl describe pod payments-api-6db7df6c9-2fmlk
```

Among the information returned, you'll find something similar to this.

```
output
Node: <none>
```

This single line tells us everything we need to know: the Pod hasn't been assigned to a worker node yet. Without a node, Kubernetes has nowhere to start the container.

Think of a Pod as a passenger waiting at an airport. The passenger has a valid ticket. The flight exists. The destination is known. But no gate has been

assigned yet. Until that happens, the passenger simply waits. Pods behave the same way.

Pod Lifecycle Before Scheduling

At this point in our deployment journey, the lifecycle looks like this.

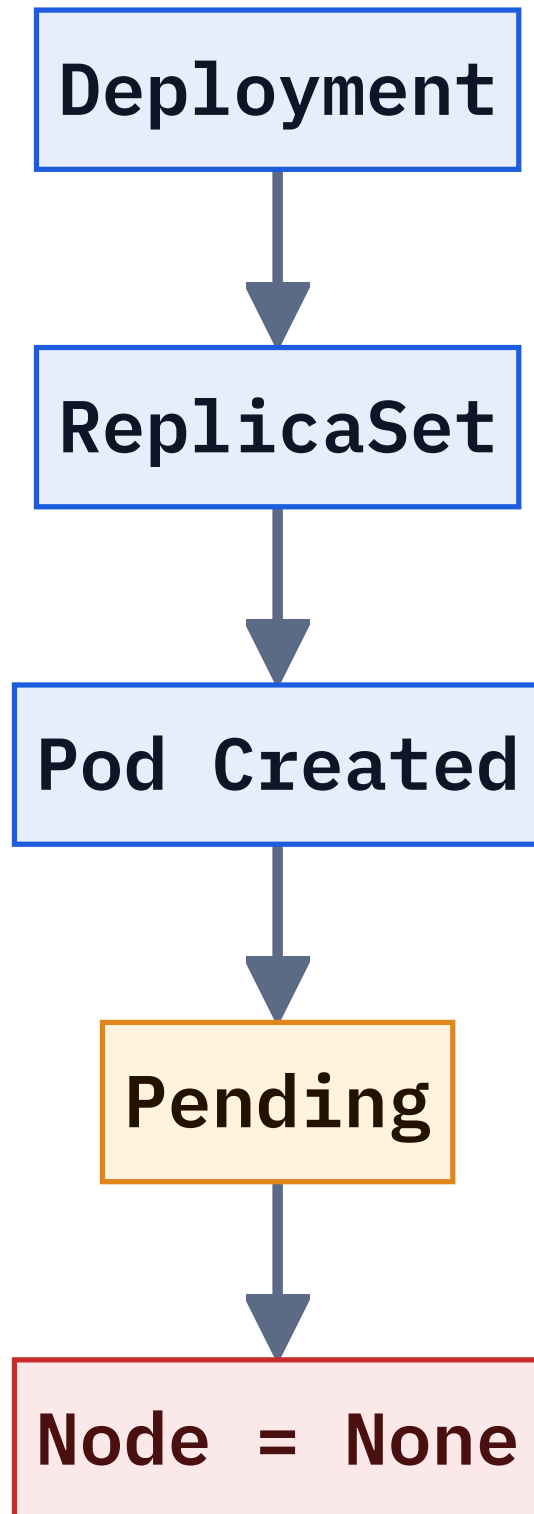


Figure 7.1 – Pod Lifecycle Before Scheduling

Nothing is wrong. Kubernetes is simply waiting for another component to make the next decision.

Production Example

One of the most common production incidents is Pods remaining in the Pending state. For example:

output		
NAME	READY	STATUS
payments-api	0/1	Pending

At this point, restarting the application won't help. Deleting the Pod won't help — the ReplicaSet will simply create another Pending Pod. The correct question isn't *"Why isn't my application starting?"* The correct question is *"Why hasn't this Pod been scheduled?"* That question leads us directly to the next Kubernetes component.

⚡ PRODUCTION INSIGHT

Whenever you encounter Pending Pods, begin by describing the Pod. Near the bottom of the output, Kubernetes usually tells you exactly why scheduling failed.

```
kubectl describe pod <pod-name> · events

Warning  FailedScheduling
0/5 nodes are available:
  3 Insufficient memory
  2 node(s) had taints that the Pod didn't tolerate
```

Learning to read these events is one of the fastest ways to diagnose scheduling issues.

✓ KEY TAKEAWAY

Pods don't run immediately after they're created. They first enter the **Pending** state while Kubernetes determines the most appropriate worker node. The ReplicaSet's job is complete; the next decision belongs to the **Kubernetes Scheduler**.

That component is the Kubernetes Scheduler. In the next chapter, we'll watch it evaluate every node in the cluster — filtering, scoring, and finally binding each Pod to the node best suited to run it.

"Which node is the best place to run this Pod?"

08 The Kubernetes Scheduler: Finding the Best Home for Every Pod

Every worker node in a Kubernetes cluster has different characteristics. Some have more CPU. Others have more memory. Some may be dedicated to production workloads, while others are reserved for machine learning jobs or GPU workloads.

When a Pod enters the Pending state, Kubernetes needs to answer one critical question: **which node is the best place to run this Pod?** That responsibility belongs to the Kubernetes Scheduler.

The Scheduler doesn't create Pods. It doesn't start containers. It doesn't communicate with worker nodes directly. Its only responsibility is to find the most suitable node for every unscheduled Pod.

Watching the API Server

Like every controller we've encountered so far, the Scheduler watches the API Server. Whenever a new Pod appears with `Node = None`, the Scheduler immediately becomes interested. It adds the Pod to its scheduling queue and begins evaluating every available worker node in the cluster.

Scheduling Is More Than Picking a Node

⚠ COMMON MISCONCEPTION

One of the biggest misconceptions is that the Scheduler simply chooses a **random** worker node. In reality, scheduling is a multi-stage decision-making process designed to place workloads where they'll perform best while respecting the constraints you've defined.

The Scheduler follows three broad phases:

- ◇ **Filtering** — eliminate nodes that cannot run the Pod.
- ◇ **Scoring** — rank the remaining nodes based on scheduling preferences.
- ◇ **Binding** — assign the Pod to the selected node.

This ensures that Pods are placed intelligently rather than randomly.

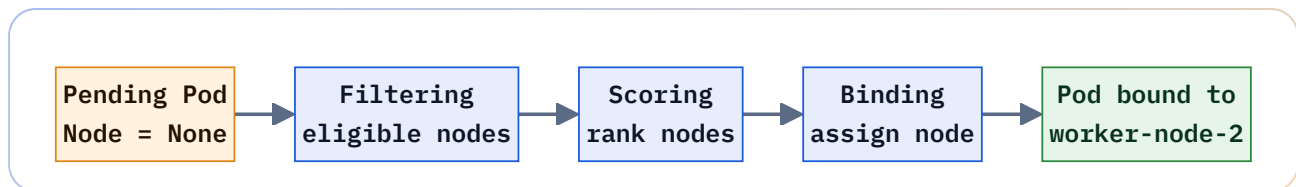


Figure 8.1 – The Scheduler's Three Phases

Filtering Nodes

The Scheduler begins by asking: *which nodes are even eligible to run this Pod?*

Nodes may be filtered out for several reasons:

- ◇ Insufficient CPU
- ◇ Insufficient memory
- ◇ Required node affinity rules
- ◇ Taints without matching tolerations

- ◇ Volume constraints
- ◇ Unsupported architectures

Any node that fails these checks is removed from consideration.

Scoring Nodes

After filtering, multiple eligible nodes may still remain. The Scheduler now ranks them based on factors such as:

- ◇ Resource utilization
- ◇ Topology spread
- ◇ Affinity preferences
- ◇ Load balancing across the cluster

The node with the highest score becomes the preferred destination for the Pod.

Binding the Pod

Once the Scheduler has selected a node, it doesn't contact the worker node directly. Instead, it sends another request to the API Server, updating the Pod's `spec.nodeName`. The Pod now looks something like this:

```
pod spec • updated by the Scheduler

spec:
  nodeName: worker-node-2
```

The Pod is still not running. But now Kubernetes knows exactly where it should run.

✓ KEY TAKEAWAY

The Scheduler doesn't create Pods or start containers — it makes a single decision: **which node is the best home for this Pod?** It filters out ineligible nodes, scores the rest, and binds the Pod by setting `spec.nodeName` through the API Server.

*The Scheduler has picked a node — so who actually starts your application? The answer is the **Kubelet**, the final component we'll explore before our application is officially running. We'll meet it in the next chapter.*

"The Scheduler decides where a Pod runs; the Kubelet ensures it actually runs."

09 The Kubelet: The Worker Node Agent

At the end of the previous chapter, the Scheduler successfully found the best worker node for each of our Pods. If we inspect one of the Pods again, we'll notice something has changed.

```
kubectl describe pod - output

Name:          payments-api-6db7df6c9-2fmlk
Namespace:     production
Node:          worker-node-2
Status:        Pending
```

The Pod is no longer waiting to be scheduled — it has been assigned to `worker-node-2`. Yet something is still missing: the application hasn't started. The Scheduler's responsibility ended the moment it assigned the Pod to a node. Now another Kubernetes component takes over: the **Kubelet**.

What Is the Kubelet?

Every worker node in a Kubernetes cluster runs a process called the Kubelet. If the API Server is the brain of the cluster, you can think of the Kubelet as the hands that carry out the work. Unlike controllers, which run in the control plane, the Kubelet runs on every worker node. Its responsibility is straightforward: **ensure that the Pods assigned to its node are actually running**.

Notice the wording carefully. The Kubelet doesn't decide which Pods should run on a node — the Scheduler already made that decision. The Kubelet's

responsibility begins *after* scheduling.

The Kubelet Watches the API Server

By now, you've probably noticed a recurring pattern throughout Kubernetes: every major component watches the API Server. The Kubelet is no different. When it starts, the Kubelet opens a watch against the API Server and listens for Pods assigned to its own node. Suppose our cluster has three worker nodes.

worker-node-1

worker-node-2

worker-node-3

The Kubelet running on `worker-node-2` ignores Pods assigned to the other two nodes. Instead, it continuously watches for Pods whose `spec.nodeName` matches its own hostname. Conceptually, the Kubelet is asking: *"Has the Scheduler assigned any new Pods to me?"* The moment our Pod is assigned to worker-node-2, the Kubelet receives that update and begins its work.

Pod Synchronization

Receiving the Pod assignment is only the beginning. The Kubelet now compares two states: the **desired state** (Pod should be running) and the **current state** (Pod does not exist on this node). The two don't match. Just like every other Kubernetes component we've studied, the Kubelet enters a reconciliation loop. Its goal is simple: make the current state on the node match the Pod specification stored in the API Server. This process is known as **Pod Synchronization**.

Putting the steps together, the Kubelet works through the following sequence to bring the Pod to life:

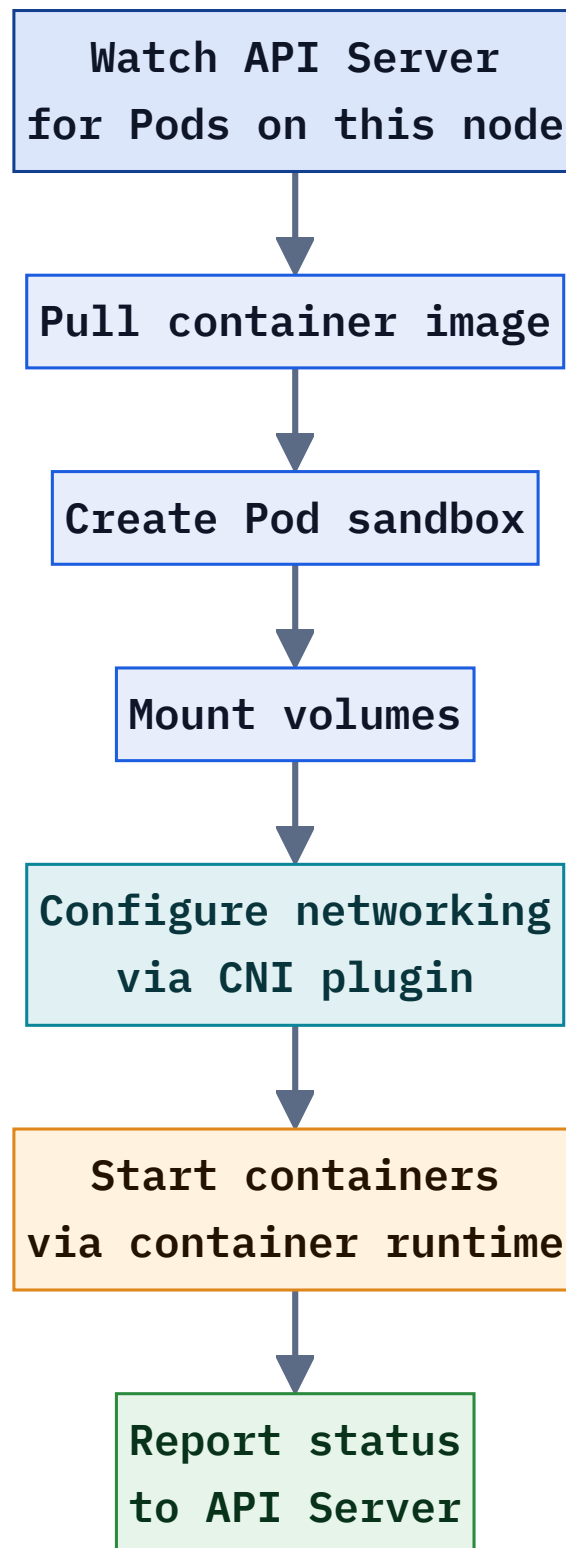


Figure 9.1 – What the Kubelet Does on the Node

Pulling the Container Image

The first task is ensuring the required container image is available. Our Deployment specifies:



```
payments-api.yaml

containers:
- name: payments-api
  image: company/payments:v1
```

The Kubelet checks whether this image already exists locally. If it does, Kubernetes skips the download. If not, it requests the container runtime to pull it from the container registry. For example, `company/payments:v1` might be downloaded from Docker Hub, Amazon ECR, Google Artifact Registry, Azure Container Registry, or another OCI-compliant registry.

Creating the Pod Sandbox

Before any containers start, Kubernetes prepares the environment in which they will run. This includes creating the **Pod sandbox** — the foundation that every container inside the Pod shares. It includes:

- ◇ The network namespace
- ◇ Shared IP address
- ◇ Hostname
- ◇ Linux namespaces
- ◇ cgroups

Only after this environment exists can the application containers be created.

Mounting Volumes

Next, the Kubelet prepares any storage required by the Pod. If the Pod references ConfigMaps, Secrets, Persistent Volumes, or EmptyDir volumes, the Kubelet mounts them before the application starts. This ensures the application sees the correct files as soon as it begins running.

Configuring Networking

The Pod still isn't ready — it doesn't yet have network connectivity. At this point, the Kubelet asks the cluster's **Container Network Interface (CNI)** plugin to configure networking. Depending on the cluster, this may be handled by solutions such as Calico, Cilium, or Flannel. The CNI plugin creates the network interface, assigns the Pod an IP address, and connects it to the cluster network.

COMMON MISCONCEPTION

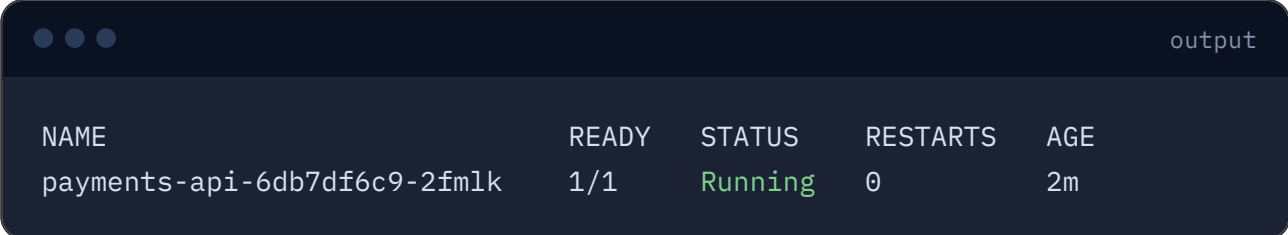
This is an important distinction: **the Kubelet requests networking; the CNI plugin provides it.** Many engineers mistakenly believe kube-proxy performs this step — it doesn't. We'll explore kube-proxy later when we discuss Services.

Starting the Containers

Everything is finally ready. The image exists. The storage has been mounted. Networking has been configured. Now the Kubelet performs its final task: it asks the **Container Runtime** to start the containers. Notice something important — the Kubelet does not start containers itself. Instead, it delegates that responsibility to another component. We'll explore that interaction in the next chapter.

Reporting Status Back to the API Server

Once the containers begin running, the Kubelet updates the Pod's status. It sends another request to the API Server indicating that the Pod has transitioned through its lifecycle. Eventually, running `kubectl get pods` shows:

A terminal window with a dark background and three window control buttons in the top-left corner. The word "output" is in the top-right corner. It displays the output of the command 'kubectl get pods'.

NAME	READY	STATUS	RESTARTS	AGE
payments-api-6db7df6c9-2fmlk	1/1	Running	0	2m

Notice what happened. The API Server didn't inspect the worker node. The Scheduler didn't verify the container started. The Kubelet is responsible for reporting the Pod's health and status back to the control plane. This keeps the cluster's view of reality synchronized with what's actually happening on the worker node.

Production Example

Suppose your Pod has already been scheduled (`Node: worker-node-2`), yet it remains stuck in `ContainerCreating` . At this point, the Scheduler has completed its work — the problem now lies on the worker node. A few common causes include:

- ◇ The image cannot be pulled.
- ◇ The storage cannot be mounted.
- ◇ The CNI plugin failed to configure networking.
- ◇ The container runtime failed to create the container.

One of the first places an SRE investigates is the Kubelet logs, or the Pod events:

```
$ journalctl -u kubelet -f  
$ kubectl describe pod payments-api
```

These events often point directly to the failing step.

⚡ PRODUCTION INSIGHT

One of the easiest ways to understand Kubernetes is to remember that the Scheduler decides *where* a Pod runs, while the Kubelet ensures it *actually* runs. When troubleshooting, always identify which component currently owns the Pod's lifecycle: if a Pod is Pending, investigate the Scheduler; if it has been assigned to a node but never starts, investigate the Kubelet. Knowing who owns the problem dramatically reduces troubleshooting time.

✓ KEY TAKEAWAY

The Kubelet is the worker node's agent. It watches the API Server for Pods assigned to its node, prepares the execution environment, requests networking and storage, and finally **delegates container creation to the container runtime.**

*We've reached the last major component in our deployment journey. The Kubelet has prepared everything; only one step remains. Who actually creates and starts the containers? The answer is the **Container Runtime** — the final component we'll explore before our application is officially running.*

"Kubernetes is a container orchestrator, not a container runtime."

10 Container Runtimes: The Final Step Before Your Application Starts

In the previous chapter, the Kubelet successfully prepared our Pod for execution. The Scheduler assigned it to a worker node. The container image was identified. Storage was prepared. Networking was configured. Everything is ready.

Yet our application still isn't running. Why? Because the Kubelet doesn't create containers. Instead, it delegates that responsibility to another component known as the **Container Runtime**. This is the final step before our application comes to life.

Why Kubernetes Doesn't Run Containers

One of the biggest misconceptions about Kubernetes is that it runs containers. It doesn't. Kubernetes is a container *orchestrator*, not a container *runtime*. Its responsibility is deciding what should run, where it should run, and when it should run. Actually creating and managing containers is someone else's responsibility.

This separation is intentional. By delegating container management to a dedicated runtime, Kubernetes can support different runtime implementations without changing the rest of the control plane. If Kubernetes were responsible for running containers itself, every improvement or bug fix related to containers would require changes to Kubernetes. Instead, Kubernetes focuses on orchestration while specialized runtimes focus on container execution.

The Container Runtime Interface (CRI)

If Kubernetes supports multiple container runtimes, how does the Kubelet communicate with all of them? The answer is the **Container Runtime Interface**, commonly known as the CRI. The CRI is a standardized interface that defines how the Kubelet communicates with a container runtime. Instead of learning how every runtime works internally, the Kubelet simply speaks the CRI. This means the Kubelet doesn't care whether your cluster is using:

containerd

CRI-O

another CRI-compatible runtime

As long as the runtime implements the CRI, the Kubelet knows how to communicate with it. Conceptually, the interaction looks like this.

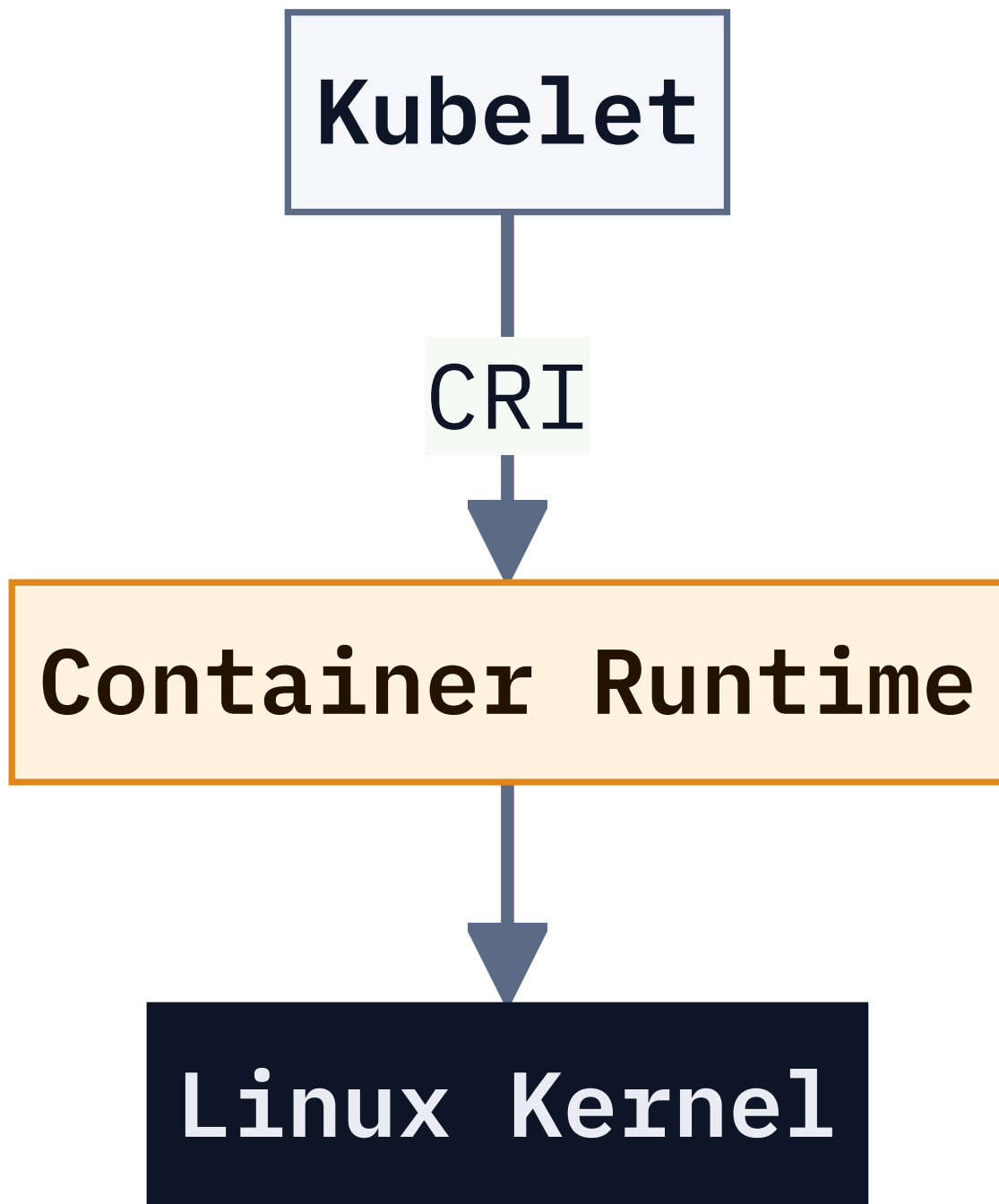


Figure 10.1 – The Kubelet Communicates Through the CRI

The CRI acts as a contract between Kubernetes and the runtime.

containerd

Today, the most commonly used container runtime in Kubernetes is **containerd**. Originally developed as part of Docker, containerd is now an independent, CNCF-graduated project designed specifically for managing container lifecycles. When the Kubelet determines that a Pod should start, it asks containerd to:

- ◇ Pull the container image if necessary.
- ◇ Create the Pod sandbox.
- ◇ Create the application containers.
- ◇ Start the containers.
- ◇ Monitor their lifecycle.
- ◇ Stop and remove containers when they are no longer needed.

Notice that Kubernetes never creates the containers directly. Everything happens through the runtime.

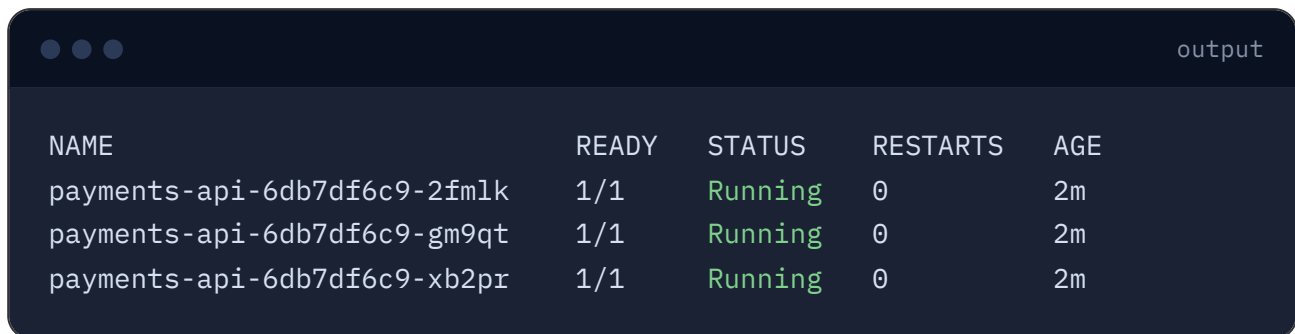
CRI-O

Another popular container runtime is **CRI-O**. Like containerd, CRI-O implements the Container Runtime Interface. Its primary goal is to provide a lightweight runtime built specifically for Kubernetes. From the Kubelet's perspective, however, nothing changes. Whether the runtime is containerd or CRI-O, the Kubelet sends exactly the same requests through the CRI. This is one of the advantages of Kubernetes' modular architecture: components can evolve independently without affecting the rest of the system.

Bringing Our Application to Life

Let's return to our Deployment. At this point, the Kubelet has finished preparing the Pod. It now sends a request to the container runtime. Conceptually, the request says: *"Create the containers described in this Pod specification."* The runtime begins executing the request. First, it checks whether the container image already exists locally. If not, it downloads the image from the configured container registry. Next, it creates the application container. Finally, it starts the container process. For the first time in our journey, the application itself begins executing.

Running `kubect1 get pods` now shows:



A terminal window with a dark background and light text. The title bar shows three dots on the left and the word 'output' on the right. The terminal displays the output of the 'kubect1 get pods' command, which is a table with five columns: NAME, READY, STATUS, RESTARTS, and AGE. There are three rows of data, all showing 'Running' status and '0' restarts.

NAME	READY	STATUS	RESTARTS	AGE
payments-api-6db7df6c9-2fmlk	1/1	Running	0	2m
payments-api-6db7df6c9-gm9qt	1/1	Running	0	2m
payments-api-6db7df6c9-xb2pr	1/1	Running	0	2m

Our application is finally running. But our journey isn't over.

Production Example

Suppose your Pod remains stuck in `ImagePullBackOff`. This tells us something important. The Scheduler has already done its job. The Kubelet has already received the Pod. The failure occurred while the runtime was attempting to pull the container image. The issue could be:

- ◇ An incorrect image name.
 - ◇ An image that doesn't exist.
 - ◇ Missing registry credentials.
 - ◇ A network connectivity issue.
-

- ◇ A private registry without the appropriate image pull secret.

Understanding where the failure occurs immediately narrows the scope of your investigation.

⚠ COMMON MISCONCEPTION

Docker is required to run Kubernetes. This was true many years ago, but it is no longer the case. Modern Kubernetes communicates with runtimes through the Container Runtime Interface (CRI). Today, most clusters use runtimes such as containerd or CRI-O. The important point is not *which* runtime is used, but that it implements the CRI.

⚙ PRODUCTION INSIGHT

When troubleshooting container startup failures, always identify which stage of the lifecycle has completed. If the Pod has no assigned node, investigate the Scheduler. If the Pod is assigned to a node but remains in ContainerCreating, investigate the Kubelet and the worker node. If the Pod reports ImagePullBackOff or ErrImagePull, focus on the container runtime, registry access, or image configuration. Breaking the deployment lifecycle into ownership boundaries helps you identify the responsible component much faster.

✓ KEY TAKEAWAY

Kubernetes doesn't run containers. The Kubelet prepares the execution environment, but it delegates container creation to a container runtime through the **Container Runtime Interface (CRI)**. By separating orchestration from container execution, Kubernetes remains flexible, modular, and capable of supporting multiple runtime implementations.

*Our deployment journey has reached an important milestone: our application is finally running. If you open the worker node, you'll find three healthy containers executing our `payments-api` application. However, there's still one major problem — **no one can reach them**. Pods are ephemeral; their IP addresses can change, and clients have no reliable way of discovering them. The next part of this publication answers a new question: how does Kubernetes make running applications discoverable and reachable?*

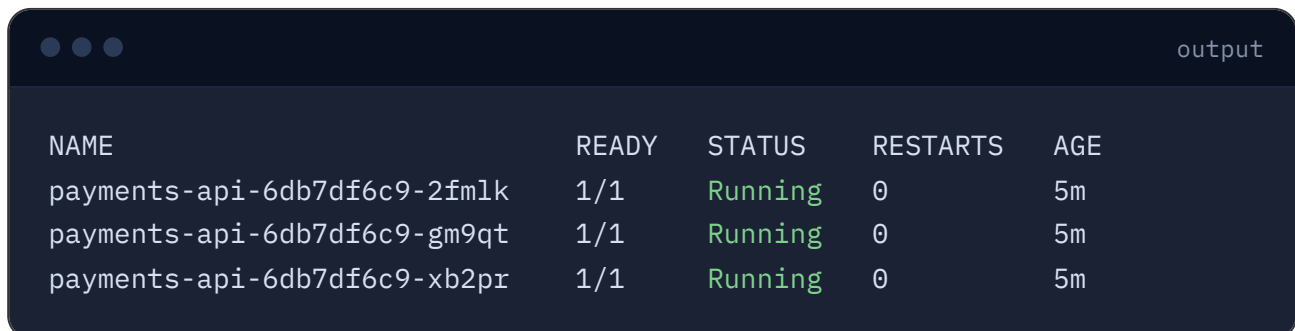
Making Your Application Reachable

The application is running — but the Pods behind it come and go. Now Kubernetes has to give clients a stable way in.

"Pods were never designed to be permanent network endpoints."

11 Your Pods Are Running... But Nobody Can Reach Them

At the end of the previous chapter, our deployment was finally complete. The Scheduler assigned our Pods to worker nodes. The Kubelet prepared the execution environment. The container runtime started our application. Running `kubectl get pods` now shows:

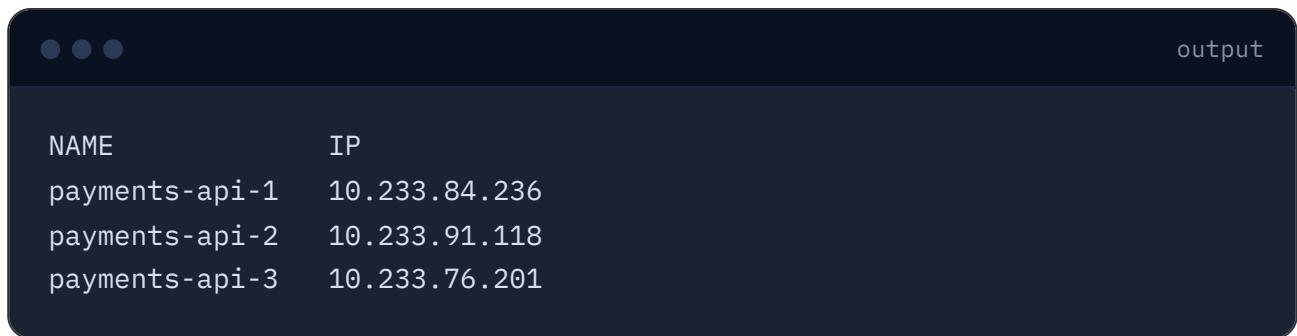
A terminal window with a dark background and light text. The title bar shows three dots on the left and the word 'output' on the right. The terminal displays the output of the 'kubectl get pods' command as a table with five columns: NAME, READY, STATUS, RESTARTS, and AGE. There are three rows of data, all showing 'Running' status and '0' restarts.

NAME	READY	STATUS	RESTARTS	AGE
payments-api-6db7df6c9-2fmlk	1/1	Running	0	5m
payments-api-6db7df6c9-gm9qt	1/1	Running	0	5m
payments-api-6db7df6c9-xb2pr	1/1	Running	0	5m

Everything looks healthy. The application is running. So we're done... right? Not quite. Imagine another application inside the cluster needs to call our Payments API. Which IP address should it use — Pod 1 (`10.233.84.236`), Pod 2 (`10.233.91.118`), or Pod 3 (`10.233.76.201`)? Immediately, another problem appears. Pods were never designed to be permanent network endpoints.

The Problem with Pod IPs

Every Pod receives its own IP address when it's created. Running `kubectl get pods -o wide` might produce:

A terminal window with a dark blue background and light blue text. The title bar on the right says "output". It displays a table with two columns: "NAME" and "IP".

NAME	IP
payments-api-1	10.233.84.236
payments-api-2	10.233.91.118
payments-api-3	10.233.76.201

At first glance, this seems useful — why not simply connect directly to those IP addresses? Because Pod IPs are temporary. A Pod exists only for as long as Kubernetes needs it. If a Pod crashes, is deleted, or is replaced during a rolling update, Kubernetes creates a brand new Pod, and that new Pod receives a completely different IP address — for example, `10.233.84.236` is deleted and replaced by `10.233.105.17`. If applications communicated directly with Pod IPs, every restart would break network communication. Clearly, Kubernetes needs a more stable solution.

Why Pods Are Ephemeral

Throughout this publication we've seen Kubernetes create and replace Pods automatically. When we deleted one of our Pods earlier, Kubernetes didn't recover it — it created an entirely new Pod. The same thing happens during rolling updates, node failures, cluster autoscaling, and application crashes. Pods are intentionally disposable; they are designed to come and go without affecting the overall application. This is one of Kubernetes' greatest strengths. However, it also creates a networking challenge: clients need a stable way to reach an application even though the Pods behind it are constantly changing.

Why Kubernetes Needs Services

This is exactly the problem that Services solve. Instead of asking clients to communicate with individual Pods, Kubernetes introduces another layer of abstraction. Clients communicate with a Service; the Service communicates with the Pods. Conceptually, it looks like this.

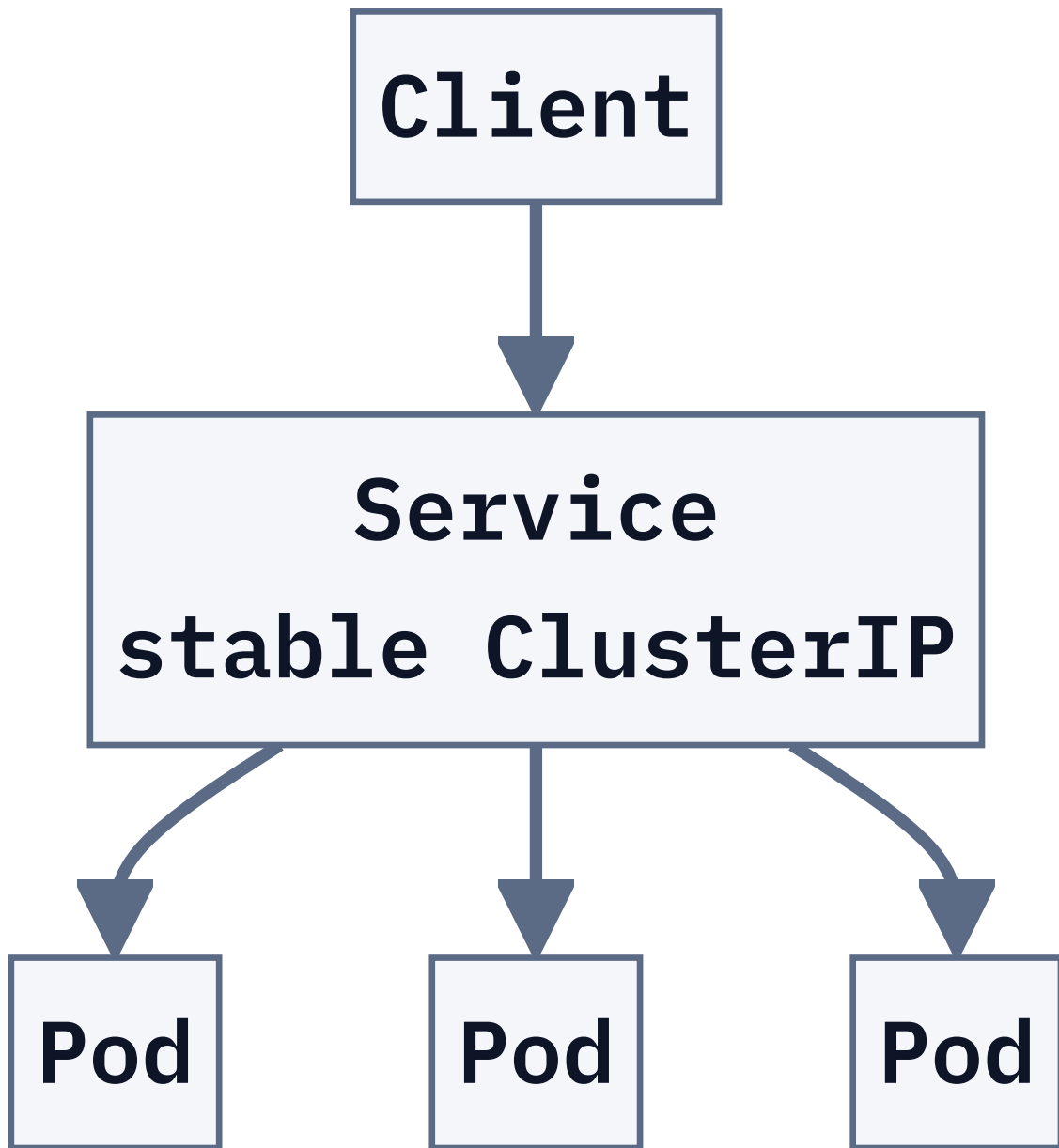


Figure 11.1 – Clients Never Talk Directly to Pods

From the client's perspective, nothing changes. Even if Kubernetes replaces every Pod behind the Service, the Service itself remains available. This gives applications a stable network identity while allowing Pods to remain ephemeral.

Production Example

Suppose your frontend application communicates directly with a Pod at `10.233.84.236`. Everything works. Now imagine the Pod crashes. The ReplicaSet immediately creates a replacement, and the new Pod receives `10.233.105.17`. The frontend is still trying to connect to `10.233.84.236`, so every request now fails. Had the frontend communicated with a Service instead, nothing would have changed — the Service would simply begin routing traffic to the replacement Pod automatically.

PRODUCTION INSIGHT

One of the easiest ways to identify Kubernetes newcomers is seeing applications configured to communicate directly with Pod IPs. In Kubernetes, Pod IPs should generally be treated as temporary implementation details. Applications should communicate through Services whenever they need a stable endpoint.

KEY TAKEAWAY

Pods are intentionally ephemeral. Their IP addresses can change at any time, making them unsuitable as permanent endpoints. **Services solve this by providing a stable network identity** that remains constant even as Pods are created, deleted, and replaced.

In the next chapter, we'll create our first Service and discover how Kubernetes gives applications a permanent address without requiring clients to know anything about the Pods behind it.

"A Service does not own Pods — it discovers them."

12 Services: Giving Your Application a Stable Identity

We've solved one problem: our application is running. Now we need a stable way for other applications to reach it. This is where Kubernetes Services come in.

A Service is one of the most important abstractions in Kubernetes. It provides a stable endpoint that clients use to communicate with an application, regardless of which Pods are currently running behind it. Notice something important: a Service does not own Pods. A Service simply **discovers** Pods and forwards traffic to them.

What Is a Service?

A Service is a Kubernetes resource that groups together Pods using labels and provides a consistent way to access them. Instead of remembering changing Pod IP addresses, clients communicate with the Service. For example:

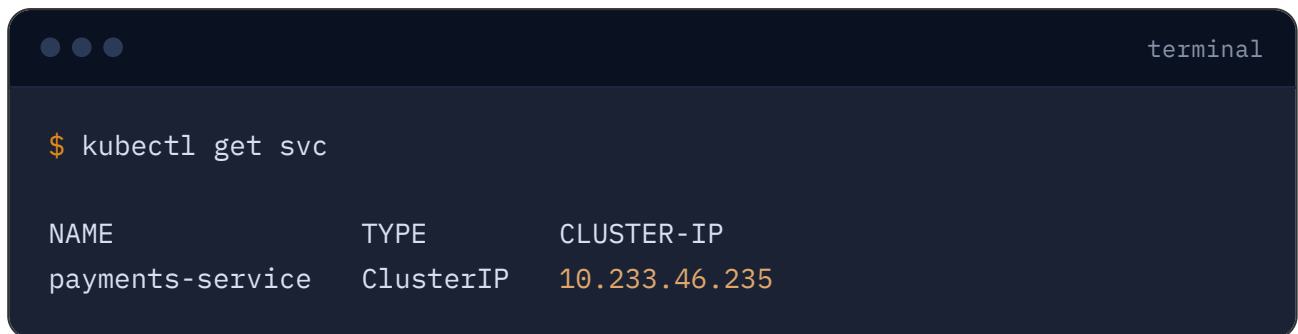
```
payments-service.yaml

apiVersion: v1
kind: Service
metadata:
  name: payments-service
spec:
  selector:
    app: payments-api
  ports:
    - port: 80
      targetPort: 8080
```


The Service doesn't specify Pod IPs. Instead, it specifies a **selector**. That selector becomes the key to everything that happens next.

ClusterIP

When the Service is created, Kubernetes automatically assigns it a **ClusterIP**. For example:

A terminal window with a dark background and light text. The title bar shows three window control buttons on the left and the word 'terminal' on the right. The command '\$ kubectl get svc' has been entered. The output is a table with three columns: NAME, TYPE, and CLUSTER-IP. The first row of data shows 'payments-service' under NAME, 'ClusterIP' under TYPE, and '10.233.46.235' under CLUSTER-IP.

```
$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP
payments-service	ClusterIP	10.233.46.235

Unlike Pod IPs, this IP remains stable throughout the lifetime of the Service. Applications communicate with this address instead of individual Pods.

Virtual IPs

The ClusterIP is often called a **Virtual IP (VIP)**. That's because it doesn't belong to a physical network interface or a Pod — it's a logical address managed by Kubernetes. When traffic is sent to the ClusterIP, Kubernetes transparently forwards the request to one of the Pods selected by the Service. Clients don't need to know which Pod handled the request.

Service Selectors

How does Kubernetes know which Pods belong to the Service? The answer is **labels**. Our Deployment assigned this label to every Pod:

```
pod labels

labels:
  app: payments-api
```

Our Service uses exactly the same selector:

```
service selector

selector:
  app: payments-api
```

This simple relationship allows Kubernetes to discover the correct Pods automatically. If a new Pod with the same label is created, it immediately becomes part of the Service. If a Pod is deleted, it is automatically removed. No manual updates are required.

Why Clients Never Talk Directly to Pods

Clients don't care which Pod serves their request — they only care that the application is available. By introducing a Service between the client and the Pods, Kubernetes can freely replace, scale, or move Pods without disrupting communication. This separation is what enables rolling updates, self-healing, and autoscaling to happen transparently.

✓ KEY TAKEAWAY

A Service gives an application a stable identity. It doesn't own Pods or maintain a list of IP addresses manually — instead, it **discovers Pods dynamically using label selectors**.

*But one question remains: where is the actual list of Pod IP addresses stored? That's the responsibility of another Kubernetes resource — **EndpointSlices** — and the subject of the next chapter.*

"The Service defines which Pods belong to it; the EndpointSlice stores their addresses."

13 EndpointSlices: Connecting Services to Pods

In the previous chapter, we created a Service for our payments-api application. The Service was assigned a stable ClusterIP (`10.233.46.235`). Clients can now send requests to that address without worrying about changing Pod IPs.

But this raises an important question. How does the Service know where to send those requests? After all, the Service only contains a selector (`app: payments-api`) — it doesn't contain any Pod IP addresses. Somewhere inside Kubernetes, there must be a list of every Pod backing this Service. That list is stored in an **EndpointSlice**.

Why EndpointSlices Exist

Think of a Service as a receptionist. The receptionist knows the company's phone number, but they don't know which employee will answer every call. Instead, they maintain a directory of employees who are currently available. EndpointSlices work the same way: a Service provides the stable address, while the EndpointSlice maintains the current list of Pods available to receive traffic. Instead of embedding Pod IPs inside the Service itself, Kubernetes stores them separately. This separation allows the list of backend Pods to change continuously without modifying the Service.

From Service Selector to Pod IP

As soon as the Service is created, another Kubernetes controller begins watching for Pods whose labels match the Service selector. When matching Pods are found, Kubernetes automatically creates an EndpointSlice containing their IP addresses. Conceptually, the process looks like this.

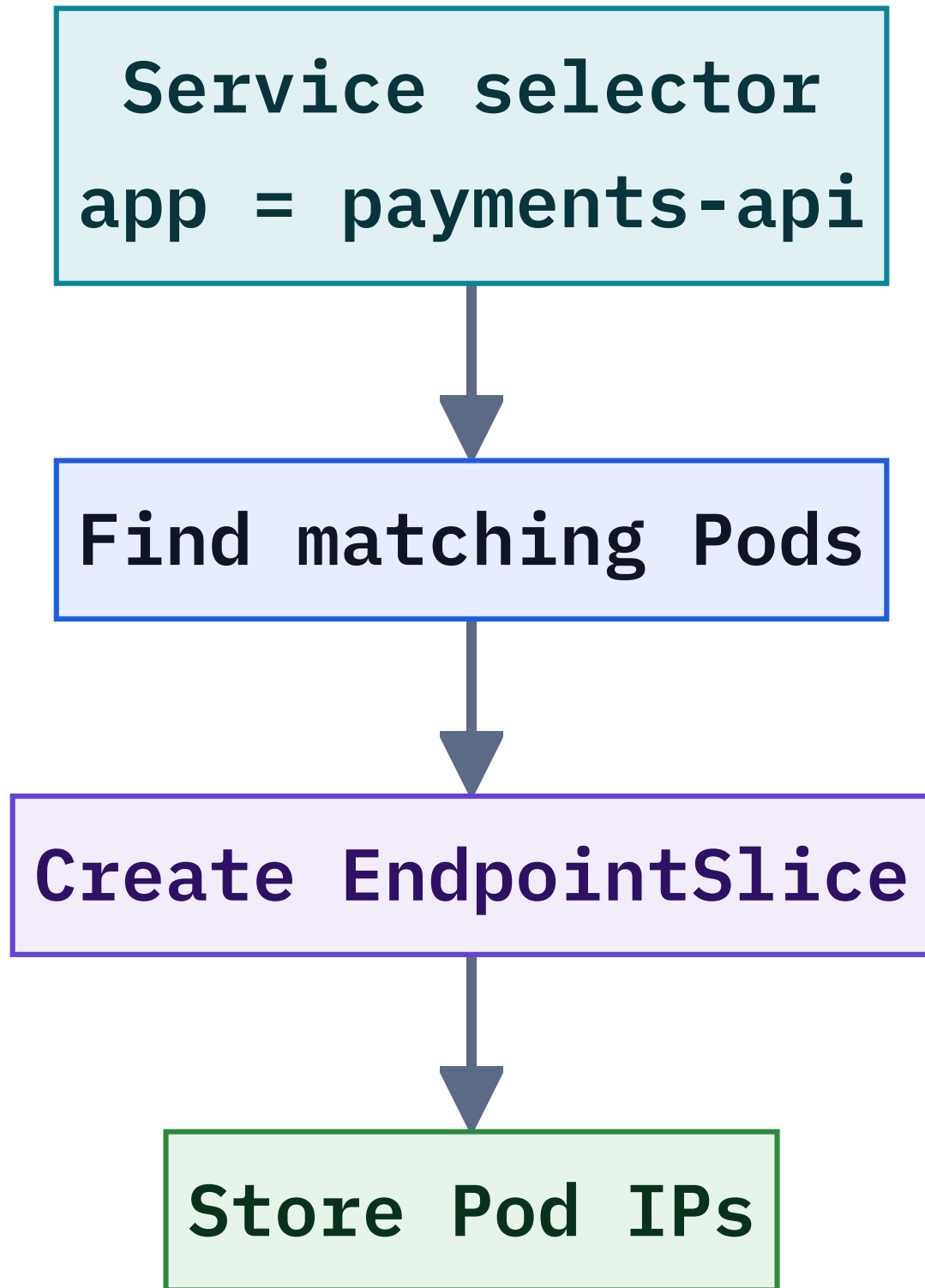


Figure 13.1 – From Labels to EndpointSlices

Notice something important: the Service never stores the Pod IPs itself. It simply defines which Pods belong to it.

How Kubernetes Knows Which Pods Belong to a Service

Suppose our Deployment currently has three running Pods, each carrying the label `app=payments-api`:

POD	LABEL	IP
payments-api-1	<code>app=payments-api</code>	10.233.84.236
payments-api-2	<code>app=payments-api</code>	10.233.91.118
payments-api-3	<code>app=payments-api</code>	10.233.76.201

Because each Pod has the matching label, the EndpointSlice automatically contains all three addresses. You can verify this yourself.

```
terminal

$ kubectl get endpointslices

NAME                      ADDRESSTYPE  PORTS  ENDPOINTS
payments-service-abcde    IPv4         8080   3

$ kubectl describe endpointslice payments-service-abcde

Endpoints:
  10.233.84.236
  10.233.91.118
  10.233.76.201
```

If you've ever run `kubectl describe service payments-service` and seen an `Endpoints:` list, those entries are coming from the EndpointSlice — the Service is simply displaying them for convenience. This is an important distinction because it explains why changes to Pods are reflected automatically without modifying the Service itself.

What Happens When Pods Change?

Suppose one Pod crashes: `kubectl delete pod payments-api-2`. Immediately, two things happen. First, the ReplicaSet notices that one replica is missing and creates a replacement Pod. Second, the EndpointSlice controller updates the EndpointSlice — the address `10.233.91.118` is replaced with the new Pod's `10.233.105.17`.

Notice what didn't change: the Service. The ClusterIP remained exactly the same. Clients continue sending traffic to the same Service address, completely unaware that one Pod has been replaced. This is one of the key reasons Kubernetes can perform self-healing without interrupting application traffic.

Why EndpointSlices Replaced Endpoints

Earlier versions of Kubernetes stored all backend Pods for a Service in a single **Endpoints** object. This worked well for small applications. However, imagine a Service managing thousands of Pods: a single Endpoints object would become very large, consuming more memory and requiring the entire object to be updated whenever a single Pod changed. EndpointSlices solve this by splitting backend Pods into multiple smaller resources. This improves scalability and reduces the amount of data that needs to be updated when Pods are added or removed. Modern Kubernetes clusters automatically use EndpointSlices, while maintaining compatibility with the older Endpoints resource.

Production Example

Imagine your application is healthy — the Pods are running, the Service exists — yet requests to the Service are failing. One of the first things an SRE should verify is whether the Service actually has any endpoints:


```
kubectl describe service payments-service

Endpoints: <none>
```

If you see `Endpoints: <none>`, the problem isn't kube-proxy — the Service currently has no healthy Pods matching its selector. Common causes include:

- ◇ The Service selector doesn't match the Pod labels.
- ◇ The Pods aren't in the Ready state.
- ◇ The Pods are running in a different namespace.
- ◇ The application hasn't passed its readiness probe.

This simple check can save a significant amount of debugging time.

⚠ COMMON MISCONCEPTION

Services store Pod IP addresses. Not exactly. Services define how to *discover* Pods through label selectors; EndpointSlices store the actual list of backend Pod IP addresses. This separation allows Kubernetes to update backend Pods dynamically without changing the Service.

✨ PRODUCTION INSIGHT

Whenever a Service isn't routing traffic correctly, don't start with kube-proxy — start with the EndpointSlice. Ask three questions: Does the Service selector match the Pod labels? Are the Pods in the Ready state? Does the EndpointSlice contain the expected Pod IPs? If the EndpointSlice is correct, the problem is likely further down the networking path. If it's empty, you've already narrowed the problem considerably.

✓ KEY TAKEAWAY

A Service provides a stable address for an application; an EndpointSlice provides the **list of Pods currently backing that Service**. Together, they allow Kubernetes to replace, scale, and heal Pods without requiring clients to know anything about changing Pod IP addresses.

*Clients know where to send requests. The EndpointSlice knows which Pods should receive them. One final question remains: who actually forwards packets from the Service's ClusterIP to one of those Pods? That responsibility belongs to **kube-proxy** — the networking component running on every worker node, and the focus of our next chapter.*

"kube-proxy programs the node so traffic to a Service reaches one of its Pods."

14 kube-proxy: From Service IP to Running Pod

At this point in our journey, our application is fully running. The Deployment created a ReplicaSet. The ReplicaSet created the Pods. The Scheduler assigned them to worker nodes. The Kubelet started the containers. The Service gave our application a stable ClusterIP. The EndpointSlice discovered the Pods backing that Service.

If we inspect the Service, we might see something like this:

```
kubectl describe service payments-service

Name:          payments-service
Type:          ClusterIP
IP:            10.233.46.235

Endpoints:
  10.233.84.236:8080
  10.233.91.118:8080
  10.233.76.201:8080
```

Everything looks correct. The Service has a stable IP. The EndpointSlice knows the Pods. But another important question remains. When an application sends traffic to `10.233.46.235`, who actually forwards that packet to one of these Pods? That responsibility belongs to **kube-proxy**.

Where kube-proxy Fits Into the Story

Throughout this publication we've seen a recurring pattern: every Kubernetes component has a single responsibility. The API Server is the single gateway

through which cluster state is persisted to etcd. The Scheduler assigns Pods to nodes. The Kubelet starts containers. EndpointSlices maintain backend Pod addresses. Likewise, kube-proxy has one responsibility: **program the node's networking so traffic sent to a Service reaches one of its backend Pods.**

Notice something important. kube-proxy does not sit in the middle of every request. It doesn't proxy traffic the way an NGINX reverse proxy does. Instead, it configures the operating system's networking so that packet forwarding happens efficiently inside the Linux kernel. This distinction is important because it explains why kube-proxy is fast even in busy production clusters.

Watching Services

Like the Scheduler, Deployment Controller, and Kubelet, kube-proxy watches the API Server. Whenever a new Service is created, kube-proxy immediately learns about it.

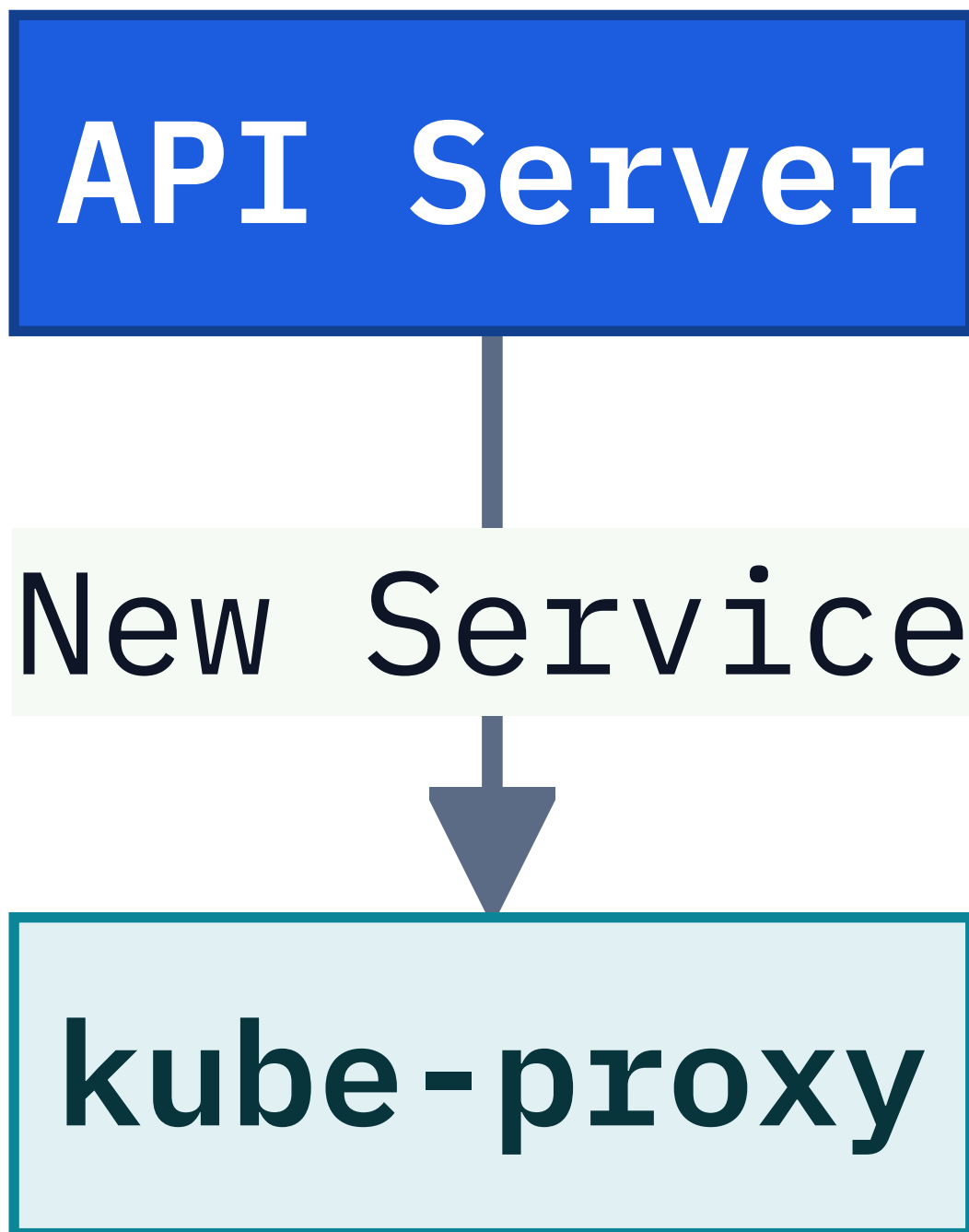


Figure 14.1 – kube-proxy Watches the API Server

Suppose we create this Service.

```
apiVersion: v1
kind: Service
metadata:
  name: payments-service
spec:
  selector:
    app: payments-api
  ports:
    - port: 80
      targetPort: 8080
```

The API Server persists the Service to etcd, and kube-proxy notices it immediately. However, kube-proxy still doesn't know where traffic should go — it only knows the Service's ClusterIP. It still needs to discover the backend Pods.

Watching EndpointSlices

This is where EndpointSlices become important. kube-proxy doesn't discover Pods by itself — instead, it watches EndpointSlices. Whenever the EndpointSlice changes, kube-proxy immediately updates its networking rules. Conceptually, the Service's ClusterIP (`10.233.46.235`) maps to an EndpointSlice listing the backend Pods `10.233.84.236` , `10.233.91.118` , and `10.233.76.201` . kube-proxy now has everything it needs: a Service IP and a list of backend Pods. Its next task is to teach Linux how to forward packets.

Programming iptables

In most Kubernetes clusters, kube-proxy operates in **iptables mode**. Rather than forwarding packets itself, kube-proxy creates a collection of iptables rules inside the Linux kernel. Those rules tell Linux something like: *if traffic arrives for `10.233.46.235` , choose one of the backend Pods and forward the packet.*

From this point onward, kube-proxy steps aside — Linux handles the packet forwarding. This is why kube-proxy isn't involved in every request after the rules have been programmed.

IPVS

Some production clusters use another mode called **IPVS**. Like iptables, IPVS forwards traffic from a Service to its backend Pods; the difference is how Linux manages those forwarding rules. IPVS was designed specifically for load balancing and performs better when clusters contain thousands of Services. From Kubernetes' perspective, nothing changes — the Service still has a ClusterIP, the EndpointSlice still contains Pod IPs. Only the underlying forwarding mechanism changes.

Packet Forwarding

Let's follow a single request. A frontend application sends an HTTP request to the Service ClusterIP. Linux consults the rules installed by kube-proxy and selects a Pod — suppose it picks `10.233.91.118`. The packet is rewritten and forwarded to that Pod. The application receives the request without ever knowing the client originally connected to the Service IP. This process happens in milliseconds.

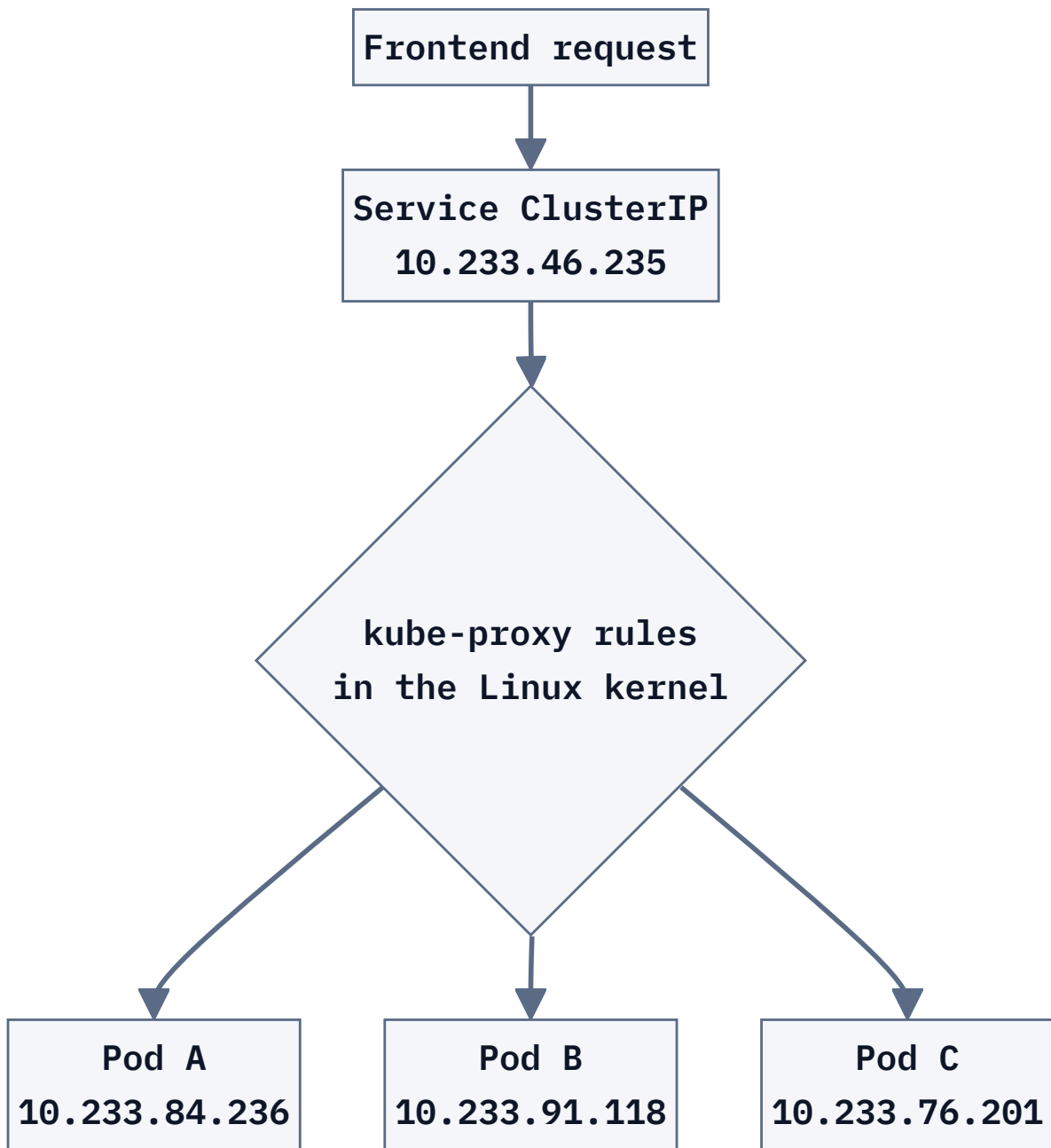


Figure 14.2 – From ClusterIP to a Backend Pod

Load Balancing Requests

Now imagine our Service has three healthy Pods. As requests arrive, kube-proxy's rules distribute traffic across those Pods.

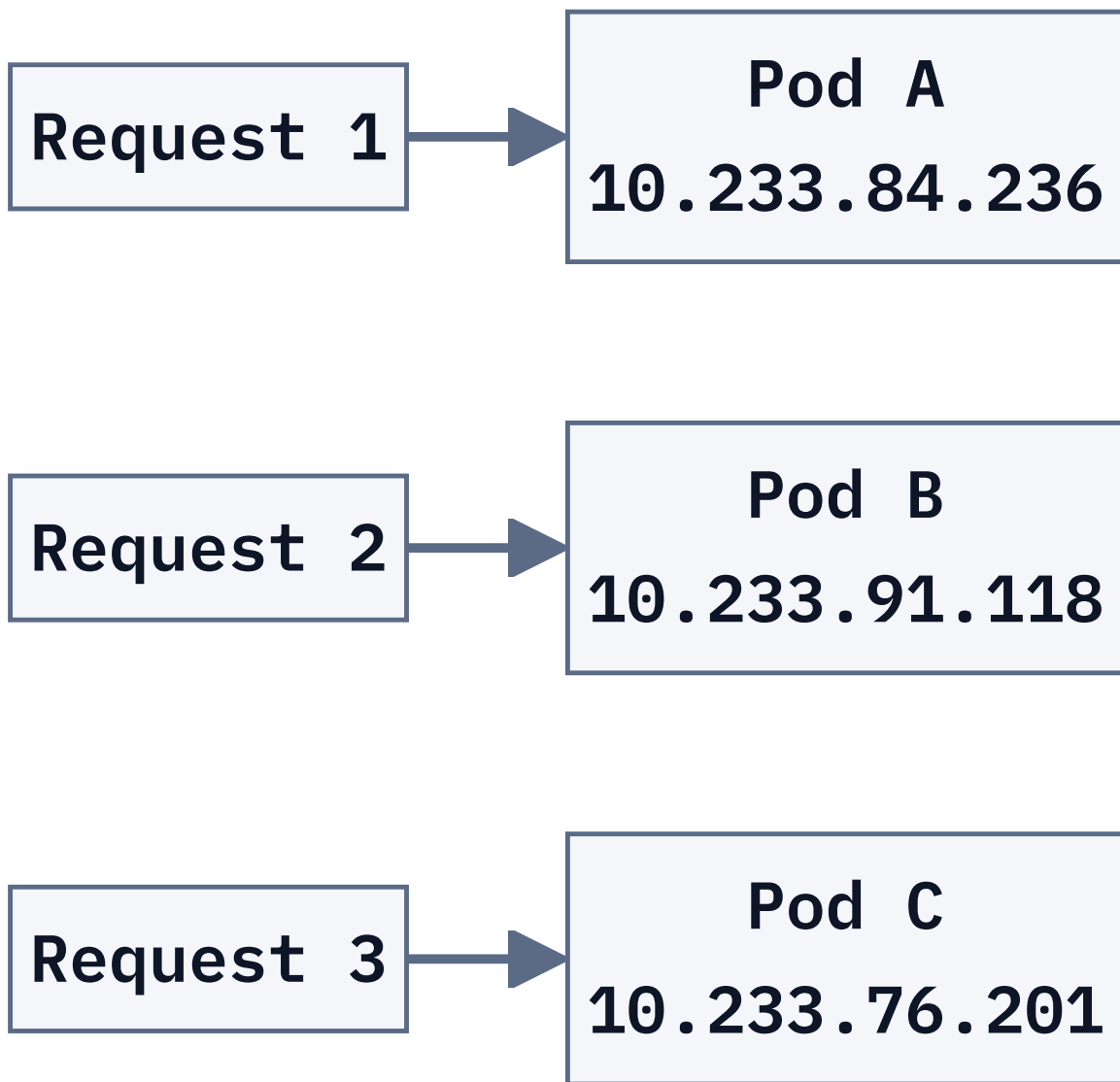


Figure 14.3 – Basic Load Balancing Across Replicas

This provides basic load balancing across the application's replicas. Exactly which Pod receives a request depends on the forwarding algorithm and the networking mode being used. The important point is that clients never choose a Pod directly — they always communicate with the Service.

Production Example

Imagine your application cannot reach another Service. Running

```
kubectl describe service payments-service
```

 shows healthy endpoints:

```
output

Endpoints:
  10.233.84.236
  10.233.91.118
  10.233.76.201
```

Everything looks correct — the EndpointSlice contains healthy Pods. At this point, the next place to investigate is kube-proxy. Common issues include:

- ◇ kube-proxy is not running on the node.
- ◇ iptables rules were not programmed correctly.
- ◇ IPVS configuration problems.
- ◇ Host firewall rules interfering with packet forwarding.

Because you've already verified the Service and EndpointSlice, you've significantly narrowed the scope of the investigation.

⚠ COMMON MISCONCEPTION

kube-proxy creates Pod networking. It doesn't. The CNI plugin creates Pod networking and assigns Pod IP addresses; kube-proxy operates *after* networking has already been established. Its responsibility is to configure how traffic reaches Services.

✦ PRODUCTION INSIGHT

When debugging Service connectivity, investigate the networking path in order:

1. Is the Pod running?
2. Is the Pod Ready?
3. Does the Service selector match the Pod labels?
4. Does the EndpointSlice contain the Pod IP?
5. Is kube-proxy running on the node?
6. Are the node's forwarding rules configured correctly?

Following this sequence prevents you from troubleshooting the wrong component.

✓ KEY TAKEAWAY

kube-proxy is the final networking component in the Service request path. It watches Services and EndpointSlices, programs the node's networking rules, and enables traffic sent to a Service's ClusterIP to **reach one of the application's Pods**.

*At this point, our application is fully reachable through a stable Service. However, Kubernetes offers another type of Service that behaves very differently: instead of hiding individual Pods behind a virtual IP, it exposes them directly. This is known as a **Headless Service**, and it's particularly useful for distributed systems like databases, message brokers, and StatefulSets, where each Pod needs its own stable network identity. We'll explore it in the next chapter.*

"Some distributed systems need to reach a specific Pod — not just any Pod."

15 Headless Services: When Your Application Needs to Reach Individual Pods

Throughout this part of the publication, we've learned that Kubernetes uses Services to hide the changing nature of Pods. Clients don't communicate directly with Pods — they communicate with a Service. The Service provides a stable ClusterIP, while kube-proxy forwards requests to one of the available Pods.

For most applications, this is exactly what we want. A frontend doesn't care which backend Pod processes its request; it only cares that the request succeeds. But not every application works this way. Some distributed systems need to communicate with a *specific* Pod, not just any Pod. This is where **Headless Services** come in.

What Changes When `clusterIP: None`?

A Headless Service looks almost identical to a regular Service. The difference is one line.

```
payments-headless.yaml

apiVersion: v1
kind: Service
metadata:
  name: payments-headless
spec:
  clusterIP: None
  selector:
    app: payments-api
  ports:
  - port: 80
    targetPort: 8080
```

That single configuration changes everything. Unlike a regular Service, Kubernetes does not assign a ClusterIP. Verify it yourself:

```
terminal

$ kubectl get svc

NAME                TYPE        CLUSTER-IP
payments-service    ClusterIP   10.233.46.235
payments-headless    ClusterIP   None
```

Notice that there is no virtual IP. This means there is no Service IP for kube-proxy to forward traffic to. So how do clients reach the Pods?

DNS Instead of a Virtual IP

Instead of returning a single ClusterIP, Kubernetes DNS returns the IP addresses of the Pods behind the Service. If an application performs a DNS lookup for the Headless Service, it receives all three addresses.

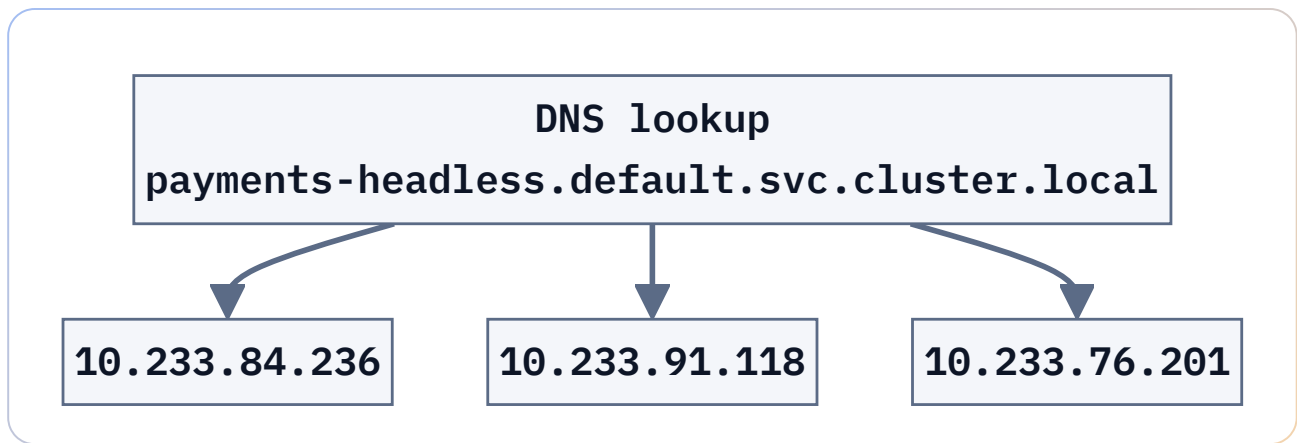


Figure 15.1 – DNS Returns Pod IPs Directly

The client can now decide which Pod to communicate with. Notice another important difference: with a regular Service, kube-proxy performs load balancing. With a Headless Service, there is no virtual IP and no Service-level load balancing — the client receives the Pod IPs directly.

StatefulSets and Stable Network Identity

Headless Services are commonly used with **StatefulSets**. Unlike Deployments, StatefulSets give every Pod a stable identity — `postgres-0`, `postgres-1`, `postgres-2` — and each Pod keeps its name even if it's recreated. Combined with a Headless Service, every Pod receives its own stable DNS record:

```
per-Pod DNS records

postgres-0.database.default.svc.cluster.local
postgres-1.database.default.svc.cluster.local
postgres-2.database.default.svc.cluster.local
```

Applications can communicate with a specific Pod instead of whichever Pod happens to receive the request. This is essential for systems where each replica has a distinct role.

Why Databases Prefer Headless Services

Many distributed databases don't want random load balancing. Consider a PostgreSQL cluster with one primary and two replicas: `postgres-0` (primary), `postgres-1` (replica), `postgres-2` (replica). If write requests were randomly distributed across all three Pods, the replicas would reject them because only the primary accepts writes. The application needs to know exactly which Pod is the primary, and a Headless Service makes this possible by exposing each Pod individually through DNS. The same pattern is common with systems such as:

Apache Kafka

Apache ZooKeeper

Cassandra

Elasticsearch

MongoDB replica sets

Redis clusters

These systems rely on direct communication between specific nodes rather than simple load balancing.

Regular Service vs Headless Service

By now, we've explored two types of Services. Although they look similar, they solve different problems.

REGULAR SERVICE	HEADLESS SERVICE
Receives a ClusterIP	No ClusterIP
kube-proxy forwards traffic	No Service-level forwarding
Clients connect to one virtual IP	Clients receive Pod IPs through DNS
Kubernetes performs load balancing	Client chooses which Pod to contact
Ideal for stateless applications	Ideal for stateful or distributed systems

The important thing to remember is that both Services still use selectors and EndpointSlices. The difference lies in how clients discover and communicate with the Pods.

Production Example

Suppose you're running a Kafka cluster. Each broker advertises its own hostname to the other brokers. If a regular Service were used, every DNS lookup would return the same ClusterIP, preventing brokers from identifying one another correctly. A Headless Service solves this problem — each broker can resolve and connect directly to its peers using stable DNS names. Without this capability, many distributed systems would fail to form or maintain a healthy cluster.

COMMON MISCONCEPTION

Headless Services give Pods static IP addresses. Not quite. Pods are still ephemeral — if a Pod is recreated, it may receive a different IP address. What remains stable is the *DNS name*, not the Pod's IP. Kubernetes updates DNS records automatically so the stable hostname always points to the Pod's current IP address.

PRODUCTION INSIGHT

Choose the type of Service based on how your application communicates. If clients simply need to reach an application and don't care which replica responds, use a regular Service. If clients need to communicate with specific Pods, or each Pod has a unique role, use a Headless Service. Understanding this distinction helps explain why Kubernetes provides both abstractions instead of forcing every application to use the same networking model.

✓ KEY TAKEAWAY

A regular Service hides individual Pods behind a stable virtual IP and lets Kubernetes distribute traffic. A **Headless Service removes that abstraction**, allowing clients to discover and communicate with individual Pods directly through DNS.

With this chapter, we've completed our networking journey. We started with running Pods that no one could reach; along the way we introduced Services, EndpointSlices, and kube-proxy to understand how Kubernetes provides stable networking despite ephemeral workloads. We've now answered another important question: how does traffic reach a running application?

In the next part, we'll return to the control plane and explore how Kubernetes keeps applications available during updates and failures — beginning with Rolling Updates.

Self-Healing and Rolling Updates

Kubernetes doesn't act once and stop. The loops that built your application never stop running.

"Updating your application without downtime."

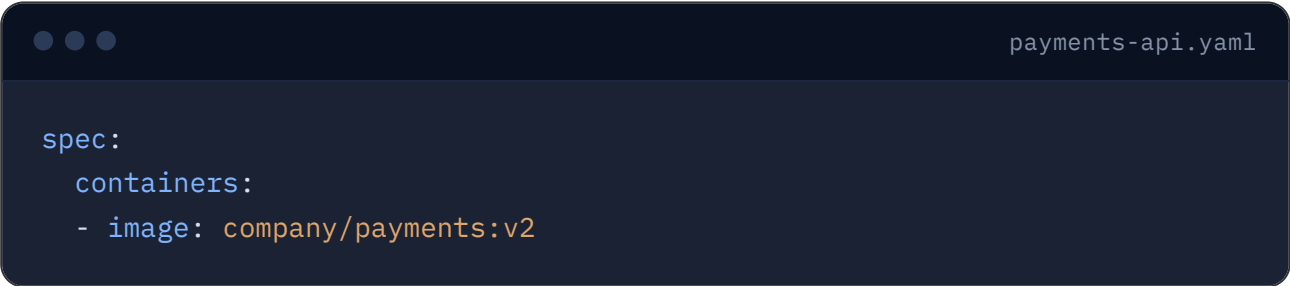
16 Rolling Updates

Up until this point, our application has been running successfully: `payments-api:v1`, three replicas, three Pods running. Users are sending requests. Everything is healthy.

A few days later, the development team releases a new version. Instead of `company/payments:v1` they want to deploy `company/payments:v2`. At first, this sounds simple — just replace the old Pods with new ones. But imagine what would happen if Kubernetes deleted all three Pods at once before creating the replacements. For a brief moment, the application would have zero running instances, and every incoming request would fail. Clearly, Kubernetes needs a safer strategy. This is where **Rolling Updates** come in.

What Happens When You Change the Image?

Let's update our Deployment.



```
spec:
  containers:
  - image: company/payments:v2
```

Apply the change, or update the image directly:

```
terminal

$ kubectl apply -f deployment.yaml

# or
$ kubectl set image deployment/payments-api \
    payments-api=company/payments:v2
```

The Deployment object is updated, and the API Server persists the new desired state to etcd. Notice what changed: the desired state is no longer `payments-api:v1` — it has become `payments-api:v2`. The current state and desired state no longer match, and that difference triggers another reconciliation cycle.

Creating a New ReplicaSet

One of the smartest design decisions in Kubernetes is that it never modifies the existing ReplicaSet. Instead, the Deployment Controller creates an entirely new ReplicaSet. If we inspect the cluster:

```
terminal

$ kubectl get rs

NAME                                DESIRED
payments-api-5fd74b66d             3
payments-api-7bd8c84fd             0
```

Notice what happened. The old ReplicaSet still exists, and a brand-new ReplicaSet has been created for the new version. This is the secret behind Kubernetes' rolling update strategy: instead of replacing Pods, Kubernetes gradually shifts traffic from one ReplicaSet to another.

Scaling Up the New ReplicaSet

Initially, the new ReplicaSet has no Pods. The Deployment Controller begins increasing its replica count while decreasing the old one. The cluster gradually transitions like this.

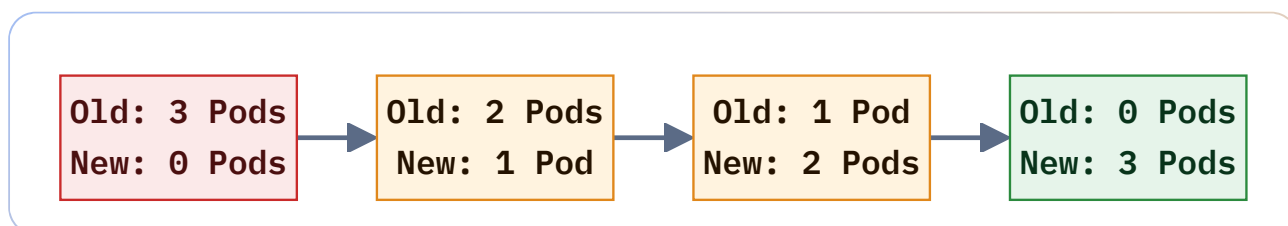


Figure 16.1 – Traffic Shifts Gradually Between ReplicaSets

At no point were all three Pods unavailable. The application remained online throughout the update.

Scaling Down the Old ReplicaSet

Notice something interesting. Kubernetes doesn't immediately remove the old ReplicaSet. Instead, it waits until replacement Pods become healthy. Only then does it reduce the number of replicas managed by the old ReplicaSet. This process continues until every old Pod has been replaced. If you watch the rollout in real time with `kubectl get pods --watch`, you'll see old Pods gradually disappear while new Pods appear. The transition is smooth enough that users often don't notice an update is happening.

Zero-Downtime Deployments

Rolling Updates are designed to keep applications available throughout the deployment process. Rather than replacing every Pod simultaneously, Kubernetes ensures that some replicas remain available while new ones are starting. The Deployment controls this behavior using two settings:

```
strategy:
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

- ◇ **maxUnavailable** — the maximum number of Pods that can be unavailable during the update.
- ◇ **maxSurge** — the maximum number of additional Pods Kubernetes can create temporarily during the rollout.

Together, they allow Kubernetes to replace application instances gradually instead of all at once.

Production Example

Suppose your application has ten replicas and you deploy version 2. Instead of deleting all ten Pods, Kubernetes performs the rollout incrementally: a new Pod is created, and once it becomes healthy and passes its readiness probe, one old Pod is removed. The process repeats until every Pod is running the new version. Users continue sending requests throughout the rollout without noticing the transition.

⚠ COMMON MISCONCEPTION

Rolling Updates replace Pods. Not exactly. Rolling Updates create an entirely new ReplicaSet. Traffic gradually shifts from the old ReplicaSet to the new one until the old ReplicaSet is no longer needed.

✧ PRODUCTION INSIGHT

One of the biggest advantages of this design is safety. If the new version fails to become healthy, Kubernetes can pause the rollout before replacing every Pod. Because the previous ReplicaSet still exists, rolling back to the earlier version is fast and straightforward. This design significantly reduces deployment risk in production environments.

✓ KEY TAKEAWAY

Rolling Updates are built on the same reconciliation principles we've seen throughout this publication. A change to the Deployment creates a new desired state. The Deployment Controller responds by creating a new ReplicaSet. The old ReplicaSet is gradually scaled down while the new one is scaled up, and **the application remains available throughout the process.**

This is another example of Kubernetes continuously working to reconcile the current state of the cluster with the desired state you've declared — a loop that, as we'll see next, never actually stops.

| *"The reconciliation loop never ends."*

17 Kubernetes Never Stops Watching

By now, you've probably noticed a recurring pattern throughout this publication. The API Server persisted our Deployment to etcd. The Deployment Controller created a ReplicaSet. The ReplicaSet created Pods. The Scheduler assigned Pods to worker nodes. The Kubelet started containers. EndpointSlices tracked Pod IP addresses. kube-proxy updated the node's networking rules.

Each component performed its task and then... waited. Or did it? Not exactly. One of the biggest misconceptions about Kubernetes is that these components perform their work once and then stop. In reality, **they never stop watching**.

Controllers Never Sleep

Every controller in Kubernetes runs continuously. Even after your application is healthy, controllers continue monitoring the cluster. They're constantly asking: *"Does the current state still match the desired state?"* If the answer is yes, they do nothing. If the answer is no, they immediately begin another reconciliation cycle. This continuous observation is what makes Kubernetes self-healing.

The Watch API

Controllers don't repeatedly poll the API Server. Instead, they maintain long-lived watch connections. Whenever something changes, the API Server notifies interested components immediately — a Deployment is updated, a

Pod is deleted, a Service changes, a Node becomes unavailable. Each relevant controller receives the update and reacts accordingly.

Informers

Rather than querying the API Server for every event, controllers rely on **Informers**. An Informer maintains a synchronized local cache of the resources a controller cares about. When an event arrives, the Informer updates its cache and notifies the controller. This allows controllers to make decisions quickly without placing unnecessary load on the API Server.

Local Cache

Think of the local cache as each controller's working copy of the cluster. Instead of asking the API Server the same question repeatedly, controllers consult their local cache — only changes are streamed from the API Server. This design keeps Kubernetes responsive even in very large clusters.

Continuous Reconciliation

Let's revisit the lifecycle of our application: Desired State → Deployment → ReplicaSet → Pods → Scheduler → Kubelet → Running Application. Now imagine a node suddenly fails and several Pods disappear. Nothing *tells* the ReplicaSet Controller to create replacements. Instead, it notices that the current state has changed, and the reconciliation loop begins again: new Pods are created, the Scheduler assigns them, the Kubelet starts them. Eventually, the cluster returns to the desired state.

A Complete Walkthrough of the Reconciliation Loop

Let's summarize the journey we've followed throughout this publication.

- ◇ The user declares a desired state.
- ◇ The API Server persists it to etcd.
- ◇ Controllers observe the new state.
- ◇ The Deployment Controller creates a ReplicaSet.
- ◇ The ReplicaSet creates Pods.
- ◇ The Scheduler assigns those Pods to worker nodes.
- ◇ The Kubelet prepares the node and starts the containers.
- ◇ Services expose the application.
- ◇ EndpointSlices maintain the list of backend Pods.
- ◇ kube-proxy forwards traffic to healthy Pods.

Every step is driven by the same idea:

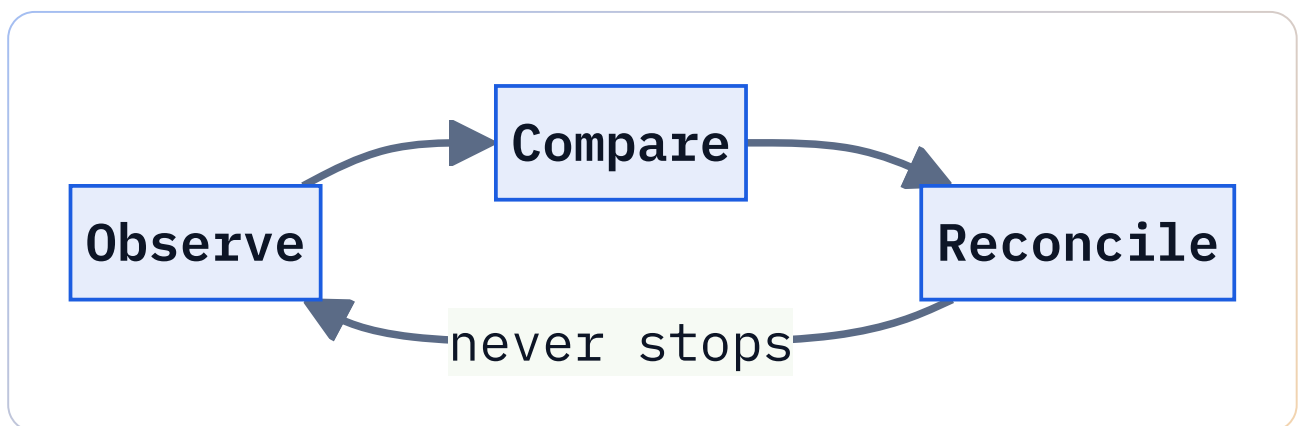


Figure 17.1 – The Heartbeat of Kubernetes

✧ PRODUCTION INSIGHT

If there's one mental model to take away from this guide, it's this: **Kubernetes is not executing a sequence of commands**. It is running a collection of independent control loops that continuously reconcile the actual state of the cluster with the desired state stored in etcd through the API Server.

Understanding that single principle makes debugging easier, because you can always ask: *which controller owns this stage of the lifecycle, and what desired state is it trying to achieve?* That question often leads you directly to the source of the problem.

✓ KEY TAKEAWAY

The components we've explored don't perform one-time actions. They continuously observe the cluster, respond to changes, and work together to keep reality aligned with the desired state. This **reconciliation loop is the foundation** of Kubernetes' resilience, self-healing, and declarative operating model.

With this chapter, we've completed the operational journey of our deployment — from a YAML file on your laptop to a resilient application running and continuously maintained inside a Kubernetes cluster.

In the final section, we turn to security: how all of these components trust one another through certificates, mutual TLS, and kubeconfig.

Establishing Trust

Every component has been talking to every other component. How do they know who they're talking to?

"How does a component know it's talking to the real API Server — and not an attacker pretending to be one?"

18 Trust Between Kubernetes Components

Throughout this publication, we've followed our Deployment from a YAML file to a running application. Along the way, we watched almost every Kubernetes component communicate with another component.

- ◇ `kubectl` sent requests to the API Server.
- ◇ The Scheduler watched the API Server for unscheduled Pods.
- ◇ The Deployment Controller watched for new Deployments.
- ◇ The Kubelet watched for Pods assigned to its node.
- ◇ `kube-proxy` watched Services and EndpointSlices.
- ◇ The API Server persisted cluster state to `etcd`.

One question remains unanswered. **How do these components know they're communicating with the real component and not an attacker pretending to be one?** In a production environment, this question is critical. If anyone could impersonate the API Server, they could modify Deployments, delete Pods, or gain access to Secrets. Likewise, if the API Server couldn't verify the identity of the Kubelet or `etcd`, it would have no way of knowing whether it was communicating with a trusted component. Kubernetes solves this problem using **mutual TLS**, often abbreviated as mTLS.

Why Security Matters

Imagine a cluster running a banking application. If an attacker could send requests to the API Server without authentication, they could delete

Deployments, modify Services, read Secrets, or schedule malicious workloads. Clearly, Kubernetes cannot trust every request it receives. Every component must first prove its identity before any communication takes place. This principle is known as **authentication**, and it forms the foundation of Kubernetes security.

Mutual TLS

Earlier in the book, we learned that `kubectl` communicates with the API Server over HTTPS. What we didn't discuss was what happens during that HTTPS connection. Unlike a typical website where only the server presents a certificate, Kubernetes often requires *both* parties to identify themselves. This process is called **mutual TLS (mTLS)**: the server proves its identity to the client, and the client proves its identity to the server. Both sides authenticate each other before exchanging any data.

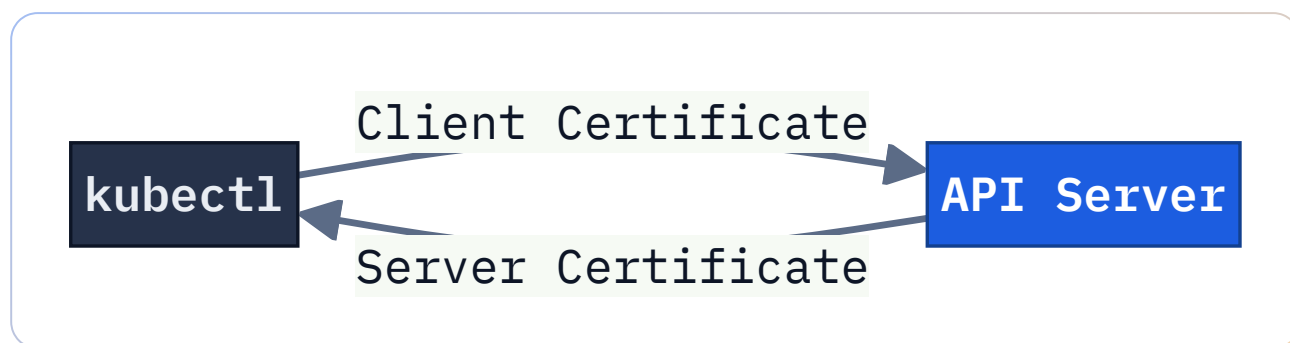


Figure 18.1 – Mutual TLS: Both Sides Prove Their Identity

Public Keys and Private Keys

Certificates rely on two cryptographic keys. Every component receives a **public key** and a **private key**. The private key remains secret and never leaves the component; the public key is included in the certificate and can be shared freely. Together, these keys allow Kubernetes components to verify identities, encrypt communication, prevent impersonation, and detect tampering. Although the underlying cryptography is complex, the important takeaway is

simple: **every trusted Kubernetes component possesses a certificate that proves its identity.**

Certificate Authorities

Of course, having a certificate raises another question. Who issued it? Anyone can create a certificate — why should Kubernetes trust it? This is where the **Certificate Authority (CA)** comes in. The CA acts as the trusted authority for the cluster, and every Kubernetes component trusts certificates signed by this CA.

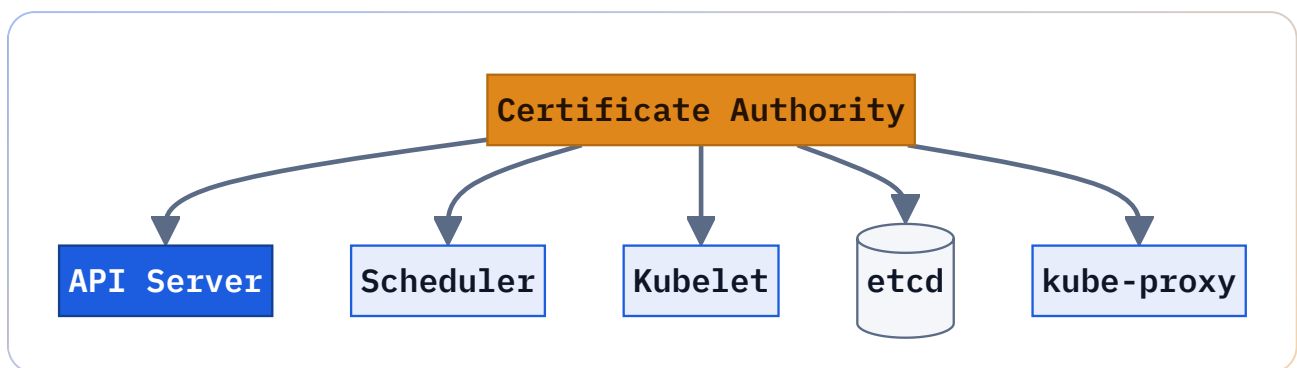


Figure 18.2 — Every Component Trusts the Cluster CA

Rather than trusting each component individually, Kubernetes trusts the CA. If the CA signed the certificate, the component is considered trustworthy.

Production Example

Imagine the API Server attempts to communicate with etcd. Before sending any data, the API Server checks etcd's server certificate. If that certificate wasn't signed by the cluster's trusted Certificate Authority, the connection fails immediately. Likewise, etcd verifies the API Server's client certificate before accepting requests. Neither component blindly trusts the other — trust must first be established.

✱ PRODUCTION INSIGHT

One of the most common causes of control plane failures is **expired certificates**. When certificates expire, components stop trusting each other. The result may include Kubelets becoming NotReady, controllers failing to communicate with the API Server, etcd refusing connections, or kubectrl returning TLS errors. Understanding certificate relationships makes these failures much easier to diagnose.

✓ KEY TAKEAWAY

Kubernetes secures communication using **mutual TLS**. Every component proves its identity through certificates signed by a trusted Certificate Authority. This allows components to communicate securely while preventing unauthorized systems from joining or impersonating members of the cluster.

In the next chapter, we'll identify exactly which Kubernetes components act as clients, which act as servers, and which certificates each one requires.

| *"Who connects to me? Who do I connect to?"*

19 Client and Server Certificates: Who Trusts Whom?

Now that we understand how Kubernetes establishes trust, let's identify the certificates used by each component. A useful way to think about certificates is to ask two questions for every component: *Who connects to me?* and *Who do I connect to?* The answer determines whether a component needs a server certificate, a client certificate, or both.

Thinking Like Kubernetes

Instead of memorizing certificates, think about the **direction of communication**. If another component connects to you, you need a **server certificate**. If you connect to another component, you need a **client certificate**. Some components do both.

API Server Certificates

The API Server is unique because it both accepts connections and initiates connections. It therefore requires a **server certificate** — so clients such as `kubectl`, the Scheduler, the Controller Manager, the Kubelet, and kube-proxy can verify they are talking to the real API Server — and a **client certificate**, for connecting securely to `etcd`. The API Server is the only control plane component that commonly acts as both a client and a server.

etcd Certificates

etcd accepts connections from the API Server, so it requires a **server certificate**. Because etcd also verifies the identity of clients connecting to it, it validates the API Server's client certificate before allowing access.

Scheduler Certificates

The Scheduler continuously watches the API Server — it initiates the connection, so it requires a **client certificate**. The Scheduler never accepts incoming connections from other Kubernetes components, so it doesn't require a server certificate.

Controller Manager Certificates

Like the Scheduler, the Controller Manager watches the API Server and submits updates through it. It acts as a client and therefore uses a **client certificate**.

Kubelet Certificates

The Kubelet performs two roles. It watches the API Server for Pods assigned to its node, and it also exposes an HTTPS endpoint used by the API Server for operations such as logs, exec, attach, and metrics. Because of these two responsibilities, the Kubelet commonly requires both a **client certificate** (to authenticate to the API Server) and a **server certificate** (to identify itself when the API Server connects back). This makes the Kubelet, like the API Server, both a client and a server.

kube-proxy Certificates

kube-proxy watches the API Server for Services and EndpointSlices. It only initiates connections, so it requires a **client certificate**.

kubectl Certificates

kubectl is simply another client of the API Server. Whenever you execute `kubectl apply -f deployment.yaml`, kubectl authenticates using the credentials stored in your kubeconfig file. Depending on the cluster configuration, these credentials may include a client certificate, a bearer token, or another supported authentication mechanism.

Certificate Summary

COMPONENT	SERVER CERTIFICATE	CLIENT CERTIFICATE
API Server	✓	✓
etcd	✓	✗
Scheduler	✗	✓
Controller Manager	✗	✓
Kubelet	✓	✓
kube-proxy	✗	✓
kubectl	✗	✓ (or another method)

Note: kubectl does not always use a client certificate. Many managed Kubernetes services authenticate kubectl using bearer tokens, cloud IAM identities, or OpenID Connect (OIDC). The important concept is that kubectl always authenticates to the API Server using one of the supported authentication methods.

✓ KEY TAKEAWAY

Instead of memorizing certificates, ask: **who connects to this component, and who does this component connect to?** Those two questions explain almost every certificate used in Kubernetes.

One question remains: how does `kubectl` know where the API Server is, and which credentials to present? That's the job of `kubeconfig`.

| *"The map that tells kubectl where to go."*

20 kubeconfig

Every time we've run a command like `kubectl apply -f deployment.yaml` or `kubectl get pods`, we've assumed that kubectl somehow knows which cluster to connect to, where the API Server is located, how to authenticate, and which Certificate Authority to trust. How does it know all of this? The answer is **kubeconfig**.

The Question kubeconfig Answers

A kubeconfig file answers four questions:

- ◇ Which cluster should I connect to?
 - ◇ Who am I?
 - ◇ Which context should I use?
 - ◇ How do I securely communicate with the API Server?
-

Without a kubeconfig file, kubectl has no idea where your cluster is or how to authenticate.

Clusters

The `clusters` section defines the API Server's address and the Certificate Authority used to verify its identity.

```
clusters:
- cluster:
  server: https://api.example.com:6443
  certificate-authority: ca.crt
```

Users

The `users` section defines how `kubectl` authenticates. Depending on the environment, this may include a client certificate and private key, a bearer token, an OpenID Connect (OIDC) configuration, or a cloud-provider authentication plugin.

Contexts

A **context** brings a cluster and a user together. It tells `kubectl`: *"Use this identity when talking to this cluster."* This allows you to manage multiple environments without changing configuration files.

Current Context

The current context determines which cluster `kubectl` uses by default.

```
$ kubectl config current-context

$ kubectl config use-context production
```

This makes it easy to work with development, staging, and production clusters from the same machine.

Mutual TLS in Practice

When you run `kubectl apply -f deployment.yaml`, the sequence is:

- ◇ `kubectl` reads the kubeconfig file.
- ◇ It identifies the target API Server.
- ◇ It verifies the API Server's certificate using the trusted CA.
- ◇ It authenticates using the configured credentials.
- ◇ A secure HTTPS connection is established.
- ◇ The Deployment request is sent to the API Server.

The kubeconfig file is therefore the bridge between your local machine and the Kubernetes control plane.

✦ PRODUCTION INSIGHT

One of the most common causes of connection issues is using the wrong context. Before troubleshooting authentication or networking, always verify which cluster you're targeting with `kubectl config current-context`. A surprising number of production incidents begin with a command executed against the wrong cluster.

✓ KEY TAKEAWAY

The kubeconfig file doesn't just store a cluster address. It defines how `kubectl` **discovers the API Server, authenticates securely, verifies the server's identity, and determines which cluster you're interacting with.**

With this chapter, we've completed the final piece of our deployment journey. We now understand not only how Kubernetes turns a YAML file into a running application, but also how every component involved in that journey establishes trust and communicates securely.

Bringing Everything Together

One Deployment. Twenty chapters. One complete story, start to finish.

"The best way to know you've truly understood Kubernetes is when you can explain exactly what happens after pressing Enter."

21 Following One Deployment From Start to Finish

Throughout this publication, we've explored Kubernetes one component at a time — the API Server, etcd, controllers, ReplicaSets, the Scheduler, the Kubelet, Services, EndpointSlices, kube-proxy, certificates, and kubeconfig. Now it's time to put everything together.

We'll follow a single Deployment from the moment you press Enter until a client successfully receives a response from your application. Think of this chapter as the complete story we've been building toward.

Step 1 The Journey Begins

Everything starts with a single command: `kubectl apply -f deployment.yaml`. Although this command appears simple, it begins a chain of events involving nearly every Kubernetes component we've explored. The first thing `kubectl` does is read your kubeconfig file, learning which cluster to connect to, where the API Server is located, how to authenticate, and which Certificate Authority to trust. Only after this information is loaded does `kubectl` establish a secure HTTPS connection to the API Server.

Step 2 The API Server Receives the Request

The API Server is the only component allowed to modify the cluster's state. Before accepting the Deployment, it performs several checks. First, it **authenticates** the client — *who are you?* Next, it **authorizes** the request — *are you allowed to create Deployments?* Finally, **Admission Controllers** validate and,

if necessary, mutate the Deployment before it is stored. Once all checks succeed, the API Server persists the Deployment to etcd. At this point, nothing is running yet.

Step 3 etcd Stores the Desired State

The Deployment now exists inside etcd, containing the Deployment name, labels, replica count, container image, resource requests, and update strategy. Notice something important: only the *desired* state has been stored. No Pods exist. No containers are running. No worker node has received any instructions.

Step 4 The Deployment Controller Wakes Up

The Deployment Controller has been watching the API Server. As soon as it detects the new Deployment, it compares the desired state (a ReplicaSet should exist) with the current state (no ReplicaSet exists). The difference triggers reconciliation, and the Deployment Controller creates a ReplicaSet.

Step 5 The ReplicaSet Creates Pod Objects

The ReplicaSet Controller immediately notices that the desired Pod count is 3 while the current count is 0. Once again, reconciliation begins. The ReplicaSet requests the creation of three Pod objects through the API Server, which persists them to etcd. Still, no containers are running.

Step 6 The Scheduler Finds a Home

Each newly created Pod has `Status: Pending` and `Node: <none>`. The Scheduler watches the API Server and notices these unscheduled Pods. It evaluates every worker node: nodes that don't satisfy scheduling

requirements are filtered out, the remaining nodes are scored, and the highest-scoring node is selected. The Scheduler binds each Pod to a worker node. Now every Pod has a home.

Step 7 The Kubelet Takes Over

The Kubelet running on each worker node notices that new Pods have been assigned to it and begins preparing the execution environment. It pulls the container image if necessary, mounts ConfigMaps, Secrets, and volumes, requests networking from the CNI plugin, and creates the Pod sandbox. Finally, it asks the Container Runtime to start the containers.

Step 8 The Container Runtime Starts the Application

The Container Runtime receives the request from the Kubelet. It downloads the image if necessary, creates the containers, and starts the application processes. For the first time, our application begins executing —

`kubectl get pods` now shows `STATUS: Running`. The application is alive. But nobody can reach it yet.

Step 9 The Service Provides a Stable Address

A Service is created for the application. Instead of communicating with changing Pod IPs, clients now communicate with a stable ClusterIP. The Service doesn't store Pod IP addresses — instead, it defines a selector.

Step 10 EndpointSlices Discover the Pods

The EndpointSlice Controller watches for Pods matching the Service selector and builds a list of backend Pod IPs (`10.233.84.236`, `10.233.91.118`,

10.233.76.201). The Service now knows which Pods belong to the application.

Step 11 kube-proxy Programs the Network

kube-proxy watches both Services and EndpointSlices, and programs the node's networking using iptables or IPVS. Whenever traffic arrives for the Service's ClusterIP, Linux forwards it to one of the backend Pods. Notice that kube-proxy doesn't *create* networking – it simply configures packet forwarding.

Step 12 The Client Sends a Request

A client sends an HTTP request.

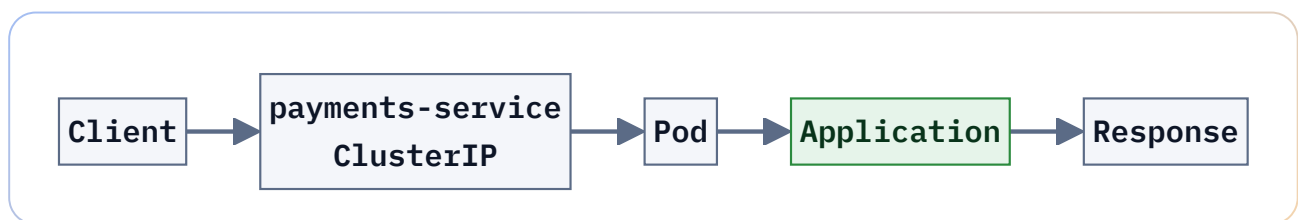


Figure 21.1 – A Request Reaches the Application

The client never needs to know which Pod handled the request, which worker node it was running on, or whether Kubernetes replaced that Pod moments earlier. The abstraction provided by Kubernetes hides all of that complexity.

Step 13 Kubernetes Keeps Watching

Even after the request succeeds, Kubernetes doesn't stop working. Every controller continues watching the cluster. If a Pod crashes, the ReplicaSet creates another. If a node fails, the Scheduler places replacement Pods elsewhere. If you deploy a new version, the Deployment Controller creates a new ReplicaSet and performs a rolling update. **Kubernetes never considers**

its work complete — it continuously reconciles the cluster so that reality matches the desired state you've declared.

The Complete Journey

This is the complete lifecycle we've followed throughout the publication — every component, in order, from a command typed on a laptop to a response served back to a client.

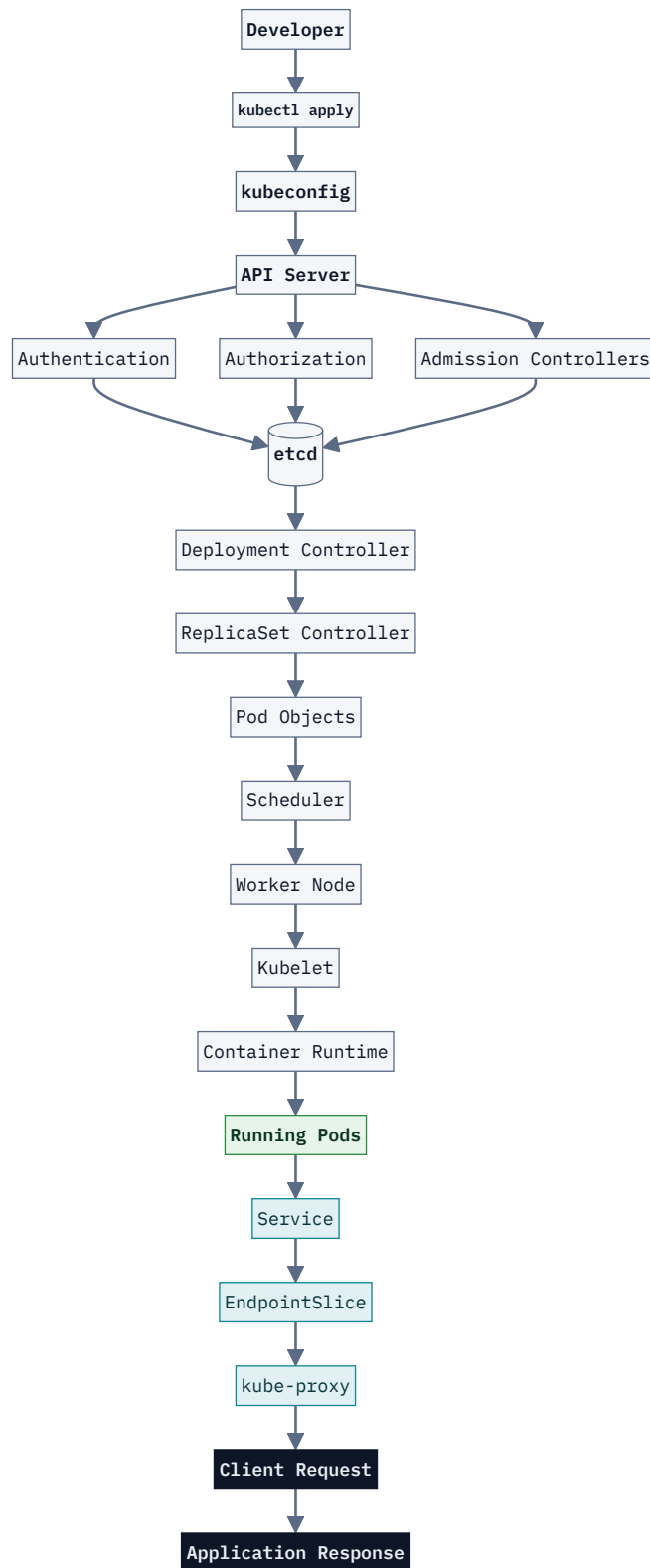


Figure 21.2 – The Complete Deployment Lifecycle

What This Journey Teaches Us

If you've made it this far, you've learned something far more valuable than a list of Kubernetes components. **You've learned how to think like Kubernetes.** Every chapter in this publication comes back to the same principle:

- ◇ A desired state is declared.
- ◇ The API Server persists it to etcd.
- ◇ Controllers observe the change.
- ◇ They reconcile the actual state with the desired state.
- ◇ Kubernetes keeps watching, ready to reconcile again whenever reality changes.

This pattern explains almost everything Kubernetes does, from scheduling and networking to rolling updates and self-healing. Once you recognize it, Kubernetes stops feeling like a collection of unrelated components and starts making sense as a coordinated system of independent control loops working together toward a common goal.

✓ KEY TAKEAWAY

Every component in this journey did exactly one thing and then handed off. No component commanded another; each simply observed the cluster and reconciled its own piece of the desired state. **That is the whole of Kubernetes** — and if you can narrate this journey, you understand it.

The reference page that follows distills every component into a single table — its one responsibility, what it watches, what it creates, and the production issue most often traced back to it.

Every component at a glance

One responsibility, one thing it watches, one thing it creates — and the failure most often traced back to it. Keep this page; it's the one to revisit long after the guide is finished.

COMPONENT	RESPONSIBILITY	WATCHES	CREATES / UPDATES	COMMON PRODUCTION ISSUE
API Server	The only component that modifies cluster state; authenticates, authorizes, admits	—	Persists every object to etcd	Unauthorized ; Forbidden ; TLS certificate error
etcd	Source of truth; stores the desired and observed state	—	Nothing — it only stores objects	Quorum loss; expired peer certificates
Deployment Controller	Ensures every Deployment has a matching ReplicaSet	Deployments	ReplicaSets	Rollout stuck; ReplicaSet never scales up
ReplicaSet Controller	Keeps the Pod count matching the desired replica count	ReplicaSets & Pods	Pod objects	Pods recreated unexpectedly; manual deletion
Scheduler	Chooses the best node for every unscheduled Pod	Pods with <code>Node: <none></code>	Sets <code>spec.nodeName</code>	Pending Pods; FailedScheduling
Kubelet	Ensures Pods assigned to its node are actually running	Pods bound to its node	Pod sandbox, volumes; reports status	ContainerCreationError; NodeNotReady
Container Runtime	Creates and starts the containers via the CRI	— (called by the Kubelet)	Containers	ImagePullBackOff; CrashLoopBackOff
CNI Plugin	Creates Pod networking	— (called by the Kubelet)	Network interface &	Pods stuck without an IP; network

COMPONENT	RESPONSIBILITY	WATCHES	CREATES / UPDATES	COMMON PRODUCTION ISSUE
	and assigns Pod IPs		Pod IP	setup failures
Service	Gives the application a stable identity (ClusterIP)	— (defines a selector)	Nothing — it discovers Pods	Selector doesn't match Pod labels
EndpointSlice Controller	Tracks which Pods currently back each Service	Services & matching Pods	EndpointSlices (Pod IPs)	Endpoints: <not ready> Pods not Ready
kube-proxy	Programs the node so Service traffic reaches a backend Pod	Services & EndpointSlices	iptables / IPVS rules	Service unreachable despite healthy endpoints
kubectl + kubeconfig	Client entry point; finds the API Server and authenticates	—	Sends requests to the API Server	Wrong context or expired credentials

Reference & Appendices

Every problem has an owner. Your job is simply to identify it.

Where should you look first?

One of the biggest advantages of understanding Kubernetes internals is that troubleshooting becomes systematic instead of guesswork. Rather than restarting Pods or searching through dozens of log files, you can identify which component owns the current stage of the application's lifecycle.

Pods Stuck in Pending

LIKELY OWNER · SCHEDULER

A Pod in the Pending state has been created, but it hasn't been assigned to a worker node.



start here

```
$ kubectl describe pod <pod-name>
```

- Insufficient CPU or memory
- Node affinity conflicts
- Taints without matching tolerations
- Unschedulable nodes

COMPONENTS INVOLVED

- Scheduler
- Nodes
- Resource Requests
- Taints & Tolerations
- Node Affinity

ImagePullBackOff

LIKELY OWNER · KUBELET / CONTAINER RUNTIME

The Pod has already been scheduled. The worker node is attempting to pull the container image but cannot.



check

```
$ kubectl describe pod <pod-name>
```

- Incorrect image name or missing tag
- Private registry credentials missing
- Registry unavailable
- Network connectivity issues

CrashLoopBackOff

LIKELY OWNER · APPLICATION

Kubernetes successfully started the container. The application itself is repeatedly crashing.



investigate

```
$ kubectl logs <pod-name>
```

- Application exceptions
- Missing environment variables or incorrect configuration
- Database connection failures
- Failed startup scripts

Services Without Endpoints

LIKELY OWNER · SERVICE SELECTOR / ENDPOINTS SLICE



verify

```
$ kubectl describe service <service-name>
```

```
Endpoints: <none>
```

- Service selector doesn't match Pod labels
- Pods in a different namespace
- Readiness probes failing

Node NotReady

LIKELY OWNER · KUBELET



verify

```
$ kubectl get nodes
```

```
$ journalctl -u kubelet -f
```

- Kubelet stopped
- Certificate issues
- Network connectivity
- Disk or memory pressure

Scheduling Failures

LIKELY OWNER · SCHEDULER / NODE CAPACITY



start with

```
$ kubectl describe pod
```

- CPU exhaustion
- Memory exhaustion
- Taints
- Affinity rules
- Persistent Volume constraints

DNS Failures

LIKELY OWNER · COREDNS



verify

```
$ kubectl exec -it <pod> -- nslookup payments-service
```

- CoreDNS healthy?
- Does the Service exist, in the right namespace?
- Network Policies blocking resolution?

Networking Failures

LIKELY OWNER · KUBE-PROXY / CNI

Always troubleshoot in this order — each step narrows the next.

- Does the Pod have an IP?
- Does the Service exist?
- Does the EndpointSlice contain the Pod IP?
- Is kube-proxy running?
- Is the CNI healthy?

Certificate Errors

LIKELY OWNER · CERTIFICATE AUTHORITY / EXPIRY

TYPICAL SYMPTOMS

```
x509: certificate signed by unknown authority
certificate has expired
```

- API Server certificate
- Kubelet certificate
- Certificate Authority & expiration dates

Authentication Errors

LIKELY OWNER · API SERVER (AUTHN / AUTHZ)

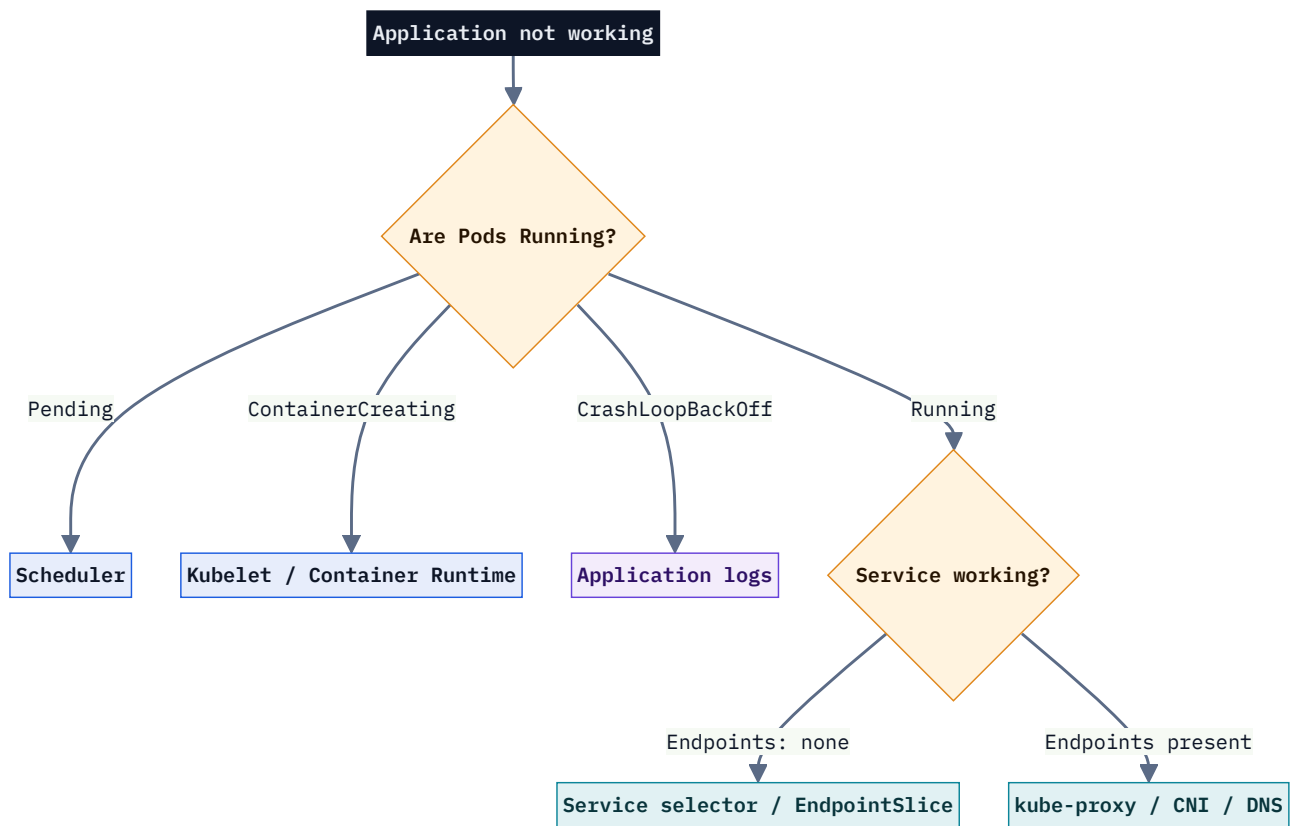
Remember the distinction — the two messages point at completely different problems.

UNAUTHORIZED	FORBIDDEN
Authentication failed. Kubernetes doesn't know who you are.	Authentication succeeded, authorization failed. Kubernetes knows who you are but won't allow the action.

Production troubleshooting flow

The mental checklist to run in production — it reinforces everything in this guide.

FIGURE BM.1 · FOLLOW THE LIFECYCLE, FIND THE OWNER



Common Kubernetes misconceptions

One goal of this publication was to replace common myths with accurate mental models. Here are the most common ones.

✕ MYTH	✓ REALITY
Deployments create Pods directly.	Deployments create ReplicaSets. ReplicaSets create Pod objects.
Kubernetes constantly polls the cluster.	Kubernetes is primarily event-driven using the Watch API.
kube-proxy creates Pod networking.	The CNI plugin creates Pod networking. kube-proxy handles Service traffic.
Services own Pods.	Services discover Pods dynamically using label selectors.
Headless Services give Pods static IP addresses.	They provide direct DNS resolution to Pods. The Pod IP may still change.
Deleting a Pod fixes the problem.	The ReplicaSet simply creates another Pod unless the underlying issue is resolved.
Kubernetes runs containers.	Kubernetes delegates container execution to a Container Runtime through the CRI.
kubeconfig is just the API Server address.	kubeconfig defines the cluster, user, context, and authentication information.
A Service stores Pod IP addresses.	EndpointSlices maintain the backend Pod IPs.
The Scheduler starts Pods.	The Scheduler only assigns Pods to nodes. The Kubelet starts them.

"Kubernetes doesn't execute commands. It reconciles desired state."

Final Thoughts

Kubernetes Is Just a Collection of Specialized Controllers

When people first learn Kubernetes, it often feels overwhelming. There are Deployments, ReplicaSets, Pods, Services, EndpointSlices, kube-proxy, the Scheduler, the Kubelet, etcd, certificates, kubeconfig, admission controllers, CRI, CNI, and many more concepts. It's easy to feel like you're memorizing dozens of unrelated components.

But after following a single Deployment from your laptop to a running application, a different picture begins to emerge. Kubernetes isn't a collection of random technologies. It's a collection of **specialized components, each with one clear responsibility**, working together to reconcile the cluster toward the desired state.

Thinking in Desired State

If there's one idea I'd like you to remember after finishing this guide, it's this:

Kubernetes doesn't execute commands. It reconciles desired state.

Every action we've explored follows the same pattern. You declare what you want. Kubernetes stores that desired state. Independent controllers observe the change. Each controller performs its own responsibility. The process repeats until the actual state matches the desired state. Once you start thinking this way, Kubernetes becomes far more predictable.

Why This Mental Model Changes Everything

This mental model changes how you troubleshoot. Instead of asking “*Why isn’t Kubernetes working?*” you begin asking “*Which component owns this stage of the lifecycle?*” That single question narrows your investigation almost immediately.

It also changes how you design applications. Rather than managing individual containers, you define the desired outcome and allow Kubernetes to maintain it continuously. That’s the power of declarative infrastructure.

Where to Go Next

Understanding the Kubernetes control plane is only the beginning. As you continue your journey, consider exploring:

Ingress Controllers & the Gateway API

StatefulSets in depth

Helm & package management

Operators & CRDs

Network Policies

Storage & Persistent Volumes

Horizontal & Vertical Pod Autoscaling

Cluster Autoscaler

GitOps with Argo CD or Flux

Observability: Prometheus, Grafana, OpenTelemetry

Service meshes: Istio, Linkerd

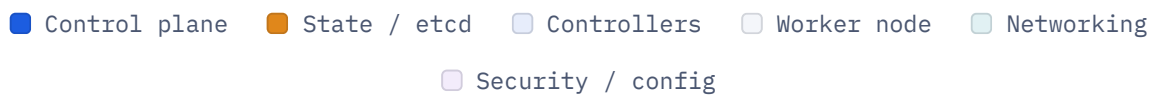
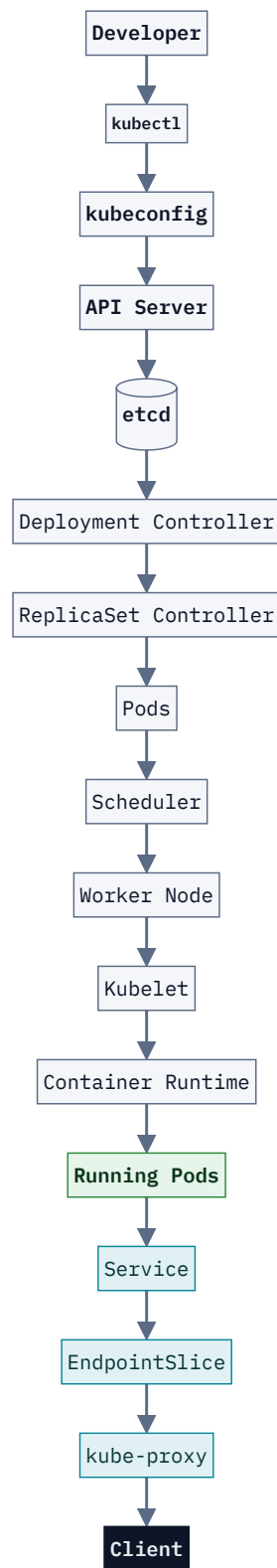
The more you understand how Kubernetes works internally, the easier these advanced topics become.

APPENDIX A

Complete Architecture Diagram

The full lifecycle on one page — the centerpiece visual of this publication.

FIGURE A.1 · DEVELOPER TO CLIENT, END TO END



Certificate Relationships

Ask two questions of every component: who connects to me, and who do I connect to?

COMPONENT	CLIENT CERTIFICATE	SERVER CERTIFICATE
API Server	✓	✓
etcd	✗	✓
Scheduler	✓	✗
Controller Manager	✓	✗
Kubelet	✓	✓
kube-proxy	✓	✗
kubectl *	Depends on authentication method	✗

* kubectl may authenticate using client certificates, bearer tokens, OIDC, or cloud IAM, depending on the cluster. The constant is that it always authenticates to the API Server by one of the supported methods.

Control Plane Components

One page, one responsibility each.

COMPONENT	PRIMARY RESPONSIBILITY
API Server	Entry point and cluster API
etcd	Source of truth
Scheduler	Assigns Pods to nodes
Controller Manager	Runs reconciliation controllers
Kubelet	Ensures Pods run on a node
Container Runtime	Creates and manages containers
kube-proxy	Programs Service networking
CNI Plugin	Provides Pod networking
CoreDNS	Service discovery and DNS

Useful kubectl Commands

Grouped by the stage of the lifecycle you're investigating.

cluster

```
$ kubectl cluster-info
$ kubectl get nodes
$ kubectl describe node <node>
```

workloads

```
$ kubectl get deployments
$ kubectl get rs
$ kubectl get pods -o wide
$ kubectl describe pod <pod>
```

networking

```
$ kubectl get svc
$ kubectl describe svc <service>
$ kubectl get endpointslices
$ kubectl describe endpointslice <name>
```

```
$ kubectl logs <pod>
$ kubectl logs -f <pod>
$ kubectl exec -it <pod> -- sh
$ kubectl get events \
    --sort-by=.metadata.creationTimestamp
```

```
$ kubectl config current-context
$ kubectl config get-contexts
$ kubectl config use-context <context>
```

Further Reading

Curated, not exhaustive — the sources worth going to first.

[Kubernetes Documentation](https://kubernetes.io/docs/home/) (https://kubernetes.io/docs/home/)

[Kubernetes Concepts](https://kubernetes.io/docs/concepts/) (https://kubernetes.io/docs/concepts/)

[Kubernetes API Reference](https://kubernetes.io/docs/reference/kubernetes-api/) (https://kubernetes.io/docs/reference/kubernetes-api/)

[etcd Documentation](https://etcd.io/docs/) (https://etcd.io/docs/)

[Container Runtime Interface \(CRI\)](https://kubernetes.io/docs/concepts/architecture/cri/) (https://kubernetes.io/docs/concepts/architecture/cri/)

[Container Network Interface \(CNI\)](https://www.cni.dev/) (https://www.cni.dev/)

[CoreDNS Documentation](https://coredns.io/manual/toc/) (https://coredns.io/manual/toc/)

[Kubernetes SIG Architecture](https://github.com/kubernetes/community/tree/master/sig-architecture) (https://github.com/kubernetes/community/tree/master/sig-architecture)

Complete. All 21 chapters across Parts I–VIII, plus the Production Debugging Guide, Common Misconceptions, Final Thoughts, and Appendices A–E — with 26 live diagrams throughout. Use the Previous / Next controls, the arrow keys, or the Table of Contents to navigate.